TestPrism: Rethinking Test Evaluation Beyond a Single Reference

Anonymous ACL submission

Abstract

Large language model (LLM) coding agents have advanced test generation across diverse programming tasks. However, the common practice of evaluating tests against a single reference solution overlooks alternative valid implementations and can overstate test quality. We introduce **TestPrism**, comprising 300 test tasks from 17 sources and 3000 candidate implementations, evenly split between valid and invalid solutions. Its primary metric, **Joint Success Function**, requires the generated tests to fail on the initial program state, accept every valid candidate, and reject every invalid candidate. Across fourteen baseline coding agent configurations, Joint Success Function reaches only 28.00%, whereas single reference success reaches 59.67%. Our analysis reveals missed behaviors, unsupported assertions, and faulty test construction. To address these weaknesses, we introduce **TestHelix**, which combines heterogeneous synthesis of test and repair pairs with peer cross validation and recursive self improvement (RSI). Across two models, TestHelix improves Joint Success Function by 8.67 to 9.00 percentage points over the native harness comparators in the TestHelix evaluation.

1 Introduction

Large language model (LLM) coding agents, such as Codex and Claude Code, can navigate repositories, modify implementations, and validate changes through execution feedback (Yang et al., 2024a; Wang et al., 2024, 2025b; Pan et al., 2026; Li et al., 2026b). These capabilities also support automated test generation and improvement (Chen et al., 2024; Alshahwan et al., 2024). Reliable tests must generalize across valid implementations while remaining sensitive to specification violations (Zhu et al., 1997; Barr et al., 2015).

Limitations of Existing Benchmarks. Test generation benchmarks span function level tasks, as in TestEval (Wang et al., 2025a), and repository issues, as in SWT Bench (Mündler et al., 2025). A common success criterion remains **single reference evaluation**: a suite must fail on the initial program and pass on one gold solution (Mündler et al., 2025; Ahmed et al., 2024). Yet a requirement can admit multiple valid implementations and plausible incorrect ones. A test can pass this check while missing required behavior (Qi et al., 2015; Liu et al., 2023) or imposing unnecessary implementation constraints. We therefore ask: *How robust is the evaluation of generated tests to variation across implementations?* Table 1 compares existing benchmarks; counting conventions and target mappings appear in Appendix A.

TestPrism: Beyond Single Reference Evaluation. TestPrism contains 300 test tasks from 17 sources across six programming languages, covering repository maintenance, algorithmic programming, and performance optimization. We adapt executable tasks to a shared interface, enrich their candidate patch sets, verify and select diverse implementations, and convert the tasks for test generation. For each test task, we retain the reference and select four valid alternatives and five invalid implementations through explicit diversity checks. Building on this pipeline, we construct TestPrism with 2,000 tasks from 17 source benchmarks, partitioned into 1,700 training tasks and 300 test tasks. The test set contains 3000 candidate implementations and spans six task categories and 34 subcategories across 67 repositories and projects. This task di-

Table 1: **Scope and evaluation targets of test generation benchmarks.**

Benchmark	Tasks	Variants	Languages	RAR	VAR	IRR	JSF	$\Delta\mathcal{C}$	EC	Scope
Block, File, and Multifile Benchmarks										
TestGenEval	1,210	✓	1	✗	✗	†	✗	✗	✗	File
CLOVER	845	✗	1	✗	✗	✗	✗	✗	✗	Block/file
MultiFileTest	60	✗	3	✗	✗	✗	✗	✗	✗	Multifile
Repository Benchmarks										
SWT Bench	1,983	✗	1	✓	✗	✗	✗	†	✗	Repo.
TDD Bench Verified	449	✗	1	✓	✗	✗	✗	†	✗	Repo.
OmniCode	396	1,109	3	✗	✗	†	✗	✗	✗	Repo.
SWE Mutation	800	2,636	10	✓	✗	†	✗	✗	✗	Repo.
TestPrism (Ours)	**300**	**2,700**	**6**	✓	✓	✓	✓	✓	✓	Block/file, Multifile, Repo.

versity supports assessing how well coding agents generalize test generation across programming scenarios. The disjoint training set supports adapting test generation strategies in TestHelix's recursive self improvement (RSI) stage.

Our evaluation protocol executes each submitted suite on the initial state and every candidate. Its primary metric, **Joint Success Function**, requires initial failure and agreement with every candidate label. Restricting the panel to the reference recovers single reference evaluation as a minimal approximation. Complementary acceptance, rejection, and coverage metrics locate behavioral disagreements and the code exercised by the tests.

TestHelix. We introduce TestHelix to address these weaknesses through heterogeneous synthesis of test and repair pairs. Its inner workflow establishes consistency within each pair, then uses peer cross validation, comprising cross execution, semantic review, and corrective feedback, to check and correct tests across pairs. Its outer RSI loop proposes test generation strategies on training tasks and validates one frozen challenger on a disjoint set before benchmark evaluation.

Takeaways. We evaluate fourteen coding agent configurations spanning six model families. Our analysis reveals four key findings: ❶ **Reliable test generation remains far from solved by frontier models.** The strongest baseline reaches only 28.00% Joint Success Function, compared with 59.67% single reference success (Table 2). Coverage tracks success: change and entry coverage correlate with Joint Success Function across configurations ($r = 0.91$ and 0.96). ❷ **Joint Success Function is a dynamic measure of reliability across implementations.** Larger candidate panels impose additional behavioral requirements, as shown in Figure 3. Single reference evaluation is its weakest special case and an upper bound on expanded panel success. ❸ **Failures span coverage gaps, faulty checks, and execution logic errors.** We sample 300 failed trajectories, 50 from the highest Joint Success Function configuration in each of six model families, and analyze 286 after excluding 14 timeout trajectories. The category shares among the analyzed trajectories are 34.6%, 50.0%, and 15.4%, respectively (Figure 4), explaining limited reliability through omissions, misjudgments, and execution failures. Missing inputs and weak assertions can pass single reference evaluation while accepting invalid patches. ❹ **Heterogeneous synthesis helps tests generalize across implementations.** With peer cross validation using diverse repairs to check and correct tests and RSI adapting test generation strategies across tasks, TestHelix raises Joint Success Function over Single by 11.67 to 12.00 percentage points under matched inference budget caps (Table 3).

Contributions. Our work makes three contributions:

- We construct **TestPrism**, with 1,700 training tasks and 300 test tasks from 17 sources, providing 3000 labeled implementations for evaluation. Its Joint Success Function criterion extends single reference evaluation to assess whether generated tests accept valid alternatives and reject incorrect implementations.
- We evaluate fourteen coding agent configurations across six model families and develop an **error taxonomy** from 286 retained failed trajectories spanning their highest Joint Success Function configurations. These complementary analyses characterize current agent performance and identify specific mistakes that cause generated tests to miss invalid behavior or reject valid alternatives, including failures hidden by single reference evaluation.
- We introduce **TestHelix**, which combines heterogeneous synthesis with peer cross validation and RSI. It checks and corrects tests through cross execution and semantic review, and adapts reusable test generation strategies to improve reliability across implementations under matched inference budget caps.

2 TestPrism

To evaluate whether generated tests generalize beyond a single reference, we construct TestPrism through four steps: **(i) task screening and adaptation**, **(ii) task selection**, **(iii) patch generation and validation**, and **(iv) diversity filtering and task conversion**. Each test instance pairs a test generation task with a curated panel of valid and invalid implementations. Figure 1 outlines the pipeline.

2.1 Step 1: Task Screening and Adaptation

We curate tasks from 17 public benchmarks, including the widely used SWE bench and Terminal Bench families (Jimenez et al., 2024; Merrill et al., 2026), requiring traceable test provenance and documented validation (Appendix B.3.1). We retain

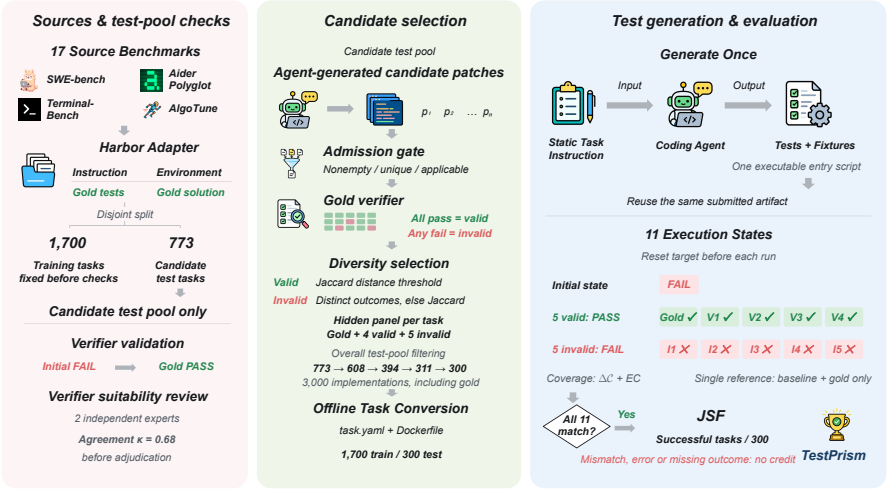


Figure 1: **Overview of TestPrism construction and evaluation.**

coding tasks whose requirements can be checked with executable tests, excluding unusable Docker images and irreproducible environments. Adapters map tasks to Harbor's instruction, environment, tests, and solution components while preserving requirements, initial states, verifiers, reference solutions, dependencies, services, and execution limits.

2.2 Step 2: Task Selection

We screen tasks using **Solution Size**, the effective lines of `solution.sh`; **Test Count**, the enumerated test functions or metadata checks; and **Test Density**, this count per file changed by the reference solution. Lower bounds are calibrated to each source's own distribution, and source quotas balance representation across the disjoint training and candidate test sets. Appendix B.2 reports the source-specific screening thresholds. We retain 1,700 training tasks for TestHelix's recursive self improvement (RSI); only candidate test tasks undergo the remaining patch generation and filtering steps.

2.3 Step 3: Patch Generation and Validation

Verifier validation. All source benchmarks provide test suites validated through execution checks, expert review, or official competition evaluation. Our additional validation round checks each adapted verifier before candidate generation, requiring failure on the initial state and success on the reference solution. Two experts use development experience to independently assess task requirements and source verifiers for potential overfitting and coverage gaps. Discussion clarifies disagreements; tasks with unresolved concerns are excluded. Independent verifier usability judgments achieve Cohen's $\kappa = 0.68$. Appendix B.3.1 documents the prior validation evidence and our additional review.

Patch generation. We generate candidate patches with different coding agents and models of varying capabilities to capture alternative solutions and errors arising during task solving, complementing mutation based fault models (Just et al., 2014; Jia and Harman, 2011). We remove empty submissions, exact duplicates, and inapplicable patches, then execute the source verifier in clean environments and record outcomes for individual checks. A candidate is **valid** if every check passes and **invalid** if any check fails. These labels remain fixed when evaluating generated tests.

2.4 Step 4: Diversity Filtering and Task Conversion

We assemble panels of one reference, four valid alternatives, and five invalid implementations. The valid group must meet the mean AST distance criterion. For invalid implementations, different sets of failed verifier checks pass the diversity check directly; matching sets require the AST Jaccard check. We refine panels through generation and replacement until both group criteria hold, then apply source quotas in final sampling.

The **test generation adapter** converts each programming task into a test authoring contract: submit tests, fixtures, and one executable entry script in a designated artifact directory. The original requirement defines the behavior to test; the candidate panel remains hidden during test generation. Appendix B.4 gives the filtering stages and implementation details.

2.5 TestPrism Statistics

TestPrism contains 2,000 tasks collected and adapted from 17 source benchmarks, with 1,700

tasks in the training set and 300 in the test set. The partitions are disjoint. The test set contains 3000 labeled implementations across six programming languages and 67 repositories and projects.

The sources span repository maintenance, evolved function specifications, olympiad programming, and numerical optimization (Xia et al., 2024; Shi et al., 2024; Press et al., 2025). Appendix B.5 provides source and language statistics, the source test count distribution (Figure 5), the six task categories and 34 subcategories, and counting conventions. Appendix B.2 details partitioning procedures and eligibility criteria. The filtering funnel appears in Appendix B.2.

2.6 Evaluation Protocol

We run each submitted entry script on the initial state and every panel implementation, resetting the target each time and recording one verdict. For task i, b_i is the initial state and g_i the reference; $\mathcal{V}_i$ contains g_i and four valid alternatives, and $\mathcal{I}_i$ contains five invalid candidates. Indicators $a_i(p)$ and $r_i(p)$ denote recorded pass and fail outcomes on p; both are zero for missing outcomes or recognized execution errors.

Behavioral metrics. **Reference Acceptance Rate** measures single reference fail to pass success over the fixed $N = 300$ tasks:

$$\mathrm{RAR} = \frac{1}{N}\sum_{i=1}^{N} r_i(b_i)\, a_i(g_i).$$

Valid Acceptance Rate measures acceptance of valid alternatives; **Invalid Rejection Rate** measures rejection of invalid candidates. For each task, A_i is the fraction of recorded valid alternatives accepted, excluding g_i, and R_i is the fraction of recorded invalid candidates rejected. Let $\mathcal{T}_V$ and $\mathcal{T}_I$ contain the tasks with defined A_i and R_i, respectively. We average each task's rate with equal weight:

$$\mathrm{VAR} = \frac{1}{|\mathcal{T}_V|}\sum_{i\in\mathcal{T}_V} A_i,$$

$$\mathrm{IRR} = \frac{1}{|\mathcal{T}_I|}\sum_{i\in\mathcal{T}_I} R_i.$$

Neither rate is gated by Reference Acceptance Rate. Missing records are excluded; recorded errors remain in the task denominators. An empty task denominator leaves its rate undefined. All full-benchmark configurations have complete observations, so both means cover all N tasks (Appendix B.6.1). We also report their harmonic mean H, computed from both rates at the precision shown in the tables.

Coverage. Following SWT Bench, **Change Coverage** ($\Delta\mathcal{C}$) measures the fraction of executable reference changes exercised by generated tests; trusted tests identify targets in the corresponding initial or reference state (Mündler et al., 2025). Following TestExplora, **Entry Coverage** measures coverage of executable code within production entry spans identified by trusted tests in the reference state, potentially beyond changed lines (Liu et al., 2026). Failed collection and empty denominators yield no numeric value; we report collection counts (Appendix B.6.2).

Joint Success Function. Our primary metric, **Joint Success Function**, takes candidate patch sets $\mathcal{P} = (P_i)_{i=1}^{N}$, where $g_i \in P_i \subseteq \mathcal{V}_i \cup \mathcal{I}_i$. It requires initial failure, acceptance of every selected valid implementation, and rejection of every selected invalid implementation:

$$\mathrm{JSF}(\mathcal{P}) = \frac{1}{N}\sum_{i=1}^{N} r_i(b_i) \prod_{p\in P_i\cap\mathcal{V}_i} a_i(p) \cdot \prod_{p\in P_i\cap\mathcal{I}_i} r_i(p).$$

Reference sets $\mathcal{P}_{\mathrm{ref}} = (\{g_i\})_{i=1}^{N}$ recover Reference Acceptance Rate. Unless otherwise specified, reported Joint Success Function values use the full panels, $P_i = \mathcal{V}_i \cup \mathcal{I}_i$, fixed across systems. Joint Success Function cannot exceed Reference Acceptance Rate; their gap measures the fraction of tasks that pass reference evaluation but fail joint evaluation, including missing or erroneous outcomes.

3 TestHelix

To improve test reliability beyond a single reference implementation, we propose TestHelix with three components (Figure 2). **Heterogeneous synthesis** generates diverse test and repair pairs to explore complementary behaviors and implementation hypotheses. **Peer cross validation** checks tests against peer repairs to expose and correct errors that agreement with one repair can miss. **Recursive self improvement (RSI)** learns from execution feedback across tasks to improve the test generation strategy.

3.1 Heterogeneous Synthesis

Diversity. We encourage diversity by assigning distinct exploration strategies to isolated coding agents. Each agent independently develops a test patch T_k and a repair patch P_k from public task

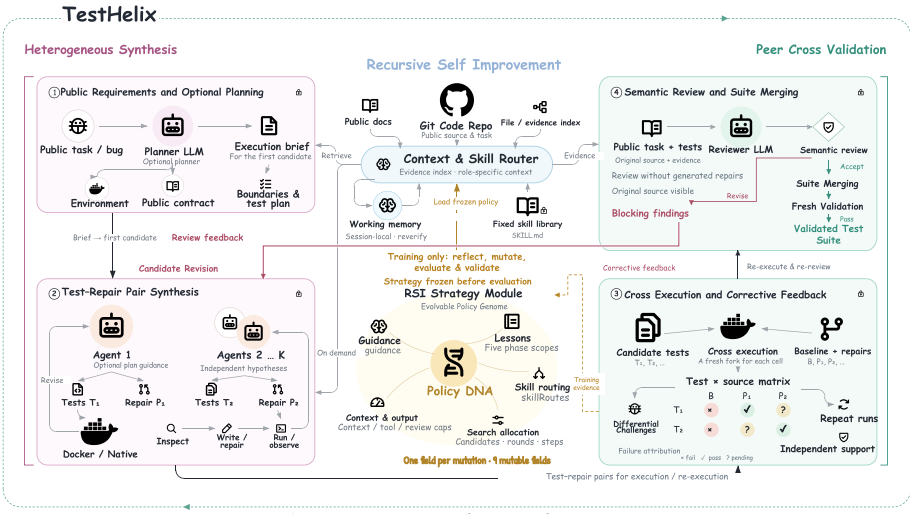


Figure 2: **TestHelix architecture.**

requirements. **Internal consistency.** We enforce internal consistency within each pair by requiring T_k to reproducibly fail on the initial program and pass on P_k. Agents use local execution feedback to refine their pairs until this condition holds or the budget is exhausted. Appendix C.1 details the synthesis prompts and branch strategies.

3.2 Peer Cross Validation

Cross execution. We execute T_i against internally consistent peer repairs P_j to expose overly restrictive tests or incomplete repairs. **Semantic review.** An independent reviewer checks assertions against public requirements without seeing generated repairs and supports every blocking finding with public evidence. **Corrective feedback.** We use reviewed counterexamples to correct the pairs. When distinct qualified repairs pass the approved suites, we use differential challenges to find inputs that separate them (Yang et al., 2011). We repeat execution and review after revision. **Suite merging.** We combine compatible suites with common passing witnesses and validate the merged suite in a fresh environment. Appendix C.2 documents the review prompt and evidence boundary.

3.3 Recursive Self Improvement

We initialize TestHelix with a basic strategy. However, improving test generation performance across heterogeneous tasks involves a large semantic search space. We use RSI to refine the strategy within the existing workflow while keeping the pair count and inference budget fixed. We optimize Reference Acceptance Rate for a denser success signal, since the stricter Joint Success Function can leave incremental harness improvements unrewarded. Heterogeneous synthesis and peer cross validation target the broader discrimination across implementations required by Joint Success Function.

We use AutoSaddler (Park et al., 2026) to optimize the harness strategy through reflective evolution. It selects a parent strategy, diagnoses its execution traces, and proposes a revision. The parent and child are evaluated on the same tasks using Reference Acceptance Rate; their performance guides selection, while reflection extracts reusable lessons for later rounds. Appendix C.3 details the optimization setup.

4 Can Coding Agents Generate Reliable Tests Across Implementations?

To assess test reliability across implementations, we investigate the following Research Questions (RQs): **RQ1** benchmarks coding agents and relates coverage to success; **RQ2** examines how expanding diverse candidate patch sets affects Joint Success Function; **RQ3** characterizes test generation failures; and **RQ4** evaluates TestHelix and its components. Evaluations follow the shared protocol in Section 2.6.

4.1 RQ1: Benchmarking Coding Agents

How reliably do current coding agents generate tests that distinguish valid and invalid implementations?

Table 2 benchmarks coding agents on all 300 tasks, reporting the highest Joint Success Function configuration from each of six model families among fourteen evaluated configurations. The main comparison uses native coding agents to reflect the close coupling between model training and harness design. Appendix D.1.1 provides the remaining eight configurations, including earlier

versions and model variants. We also examine how coverage and the balance between acceptance and rejection relate to Joint Success Function across all fourteen configurations; Appendix D.1.2 provides the correlation analysis.

Table 2: **Performance of the highest Joint Success Function configuration from each model family.**

Model	Reference	Candidates			Coverage		Joint
	RAR	VAR	IRR	H	ΔC	EC	JSF
Claude Fable 5.1	**59.67%**	72.67%	79.33%	**75.85%**	82.33%	**54.33%**	**28.00%**
Kimi K3	56.67%	63.67%	83.00%	72.06%	**84.33%**	52.00%	24.67%
Qwen3.8 Max	54.67%	**77.00%**	66.00%	71.08%	80.33%	50.33%	23.67%
GPT 5.6 sol -max	54.00%	61.33%	86.00%	71.60%	70.33%	49.00%	23.33%
DeepSeek V4 Pro	41.00%	57.67%	70.33%	63.37%	67.33%	43.33%	13.33%
GLM 5.3	43.00%	50.33%	**86.33%**	63.59%	64.67%	44.33%	11.33%

Reference success often fails to extend to other implementations. Claude Fable 5.1 leads Joint Success Function at 28.00%, compared with 59.67% against the reference. Five of the six configurations score higher on Invalid Rejection Rate than Valid Acceptance Rate, indicating difficulty accepting permissible implementations. GLM 5.3 has the highest rejection rate but the lowest acceptance and joint success rates: defect rejection alone is insufficient. Across all fourteen configurations, the positive association between H and Joint Success Function supports balancing both requirements. Change Coverage and Entry Coverage also correlate with joint success (Pearson $r = 0.91$ and 0.96). Reaching relevant behavior helps, but does not ensure that assertions judge it correctly (Inozemtseva and Holmes, 2014; Zhu et al., 1997).

> **Answer to RQ1: Reliable test generation remains far from solved by frontier models.** The highest baseline Joint Success Function is only 28.00%. Valid Acceptance Rate and Invalid Rejection Rate expose complementary weaknesses in accepting valid alternatives and rejecting defective patches; their harmonic mean H and coverage both track joint success.

4.2 RQ2: Patch Diversity and Joint Success

How does expanding a diverse candidate patch set affect Joint Success Function?

Figure 3 shows Joint Success Function and candidate diversity under three patch addition schedules. We summarize diversity using mean pairwise Jaccard distance between AST feature sets. For this diagnostic, we randomly sample 50 benchmark tasks and continue the candidate construction pipeline to expand each task's standard ten implementations to 15: the reference, seven additional valid alternatives, and seven invalid candidates. We keep each submitted suite fixed and grow its candidate set from the reference repair by adding valid alternatives, invalid candidates, or the two in alternation. The alternating schedule starts with an invalid addition and contains eight valid and seven invalid implementations at $k = 15$. The diagnostic equally weights the six model configurations in Table 2, each evaluated on the same 50 tasks. Candidate count and diversity change together in this diagnostic; Appendix D.2 provides the construction and calculation details.

The results show two ways in which an expanded panel can remove reference successes: tests may reject valid alternatives or accept invalid candidates. Mean Joint Success Function starts at 49.33%, reaches 38.10% after five valid additions and 30.49% after five invalid additions, and reaches 24.33% after nine alternating additions. The complete pool endpoints are 37.33%, 29.33%, and 23.33% for the valid, invalid, and alternating schedules. The tests remain fixed while the panel imposes additional acceptance and rejection require-

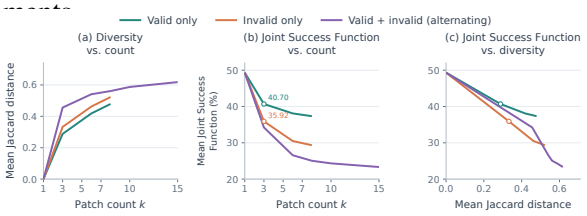


Figure 3: Joint Success Function and candidate diversity under three patch addition schedules.

> **Answer to RQ2: Diverse patch additions can expose test errors left unchecked by smaller panels.** Joint Success Function depends on which implementations a panel contains as well as their number. Expanding the panel checks acceptance of valid alternatives and rejection of defective candidates. Using only the reference reduces the criterion to Reference Acceptance Rate.

4.3 RQ3: Analysis of Test Generation Failures

What error mechanisms occur in generated tests, and which remain hidden under single reference evaluation?

We sample 50 failed trajectories, each with a zero task-level joint success indicator, from each configuration in Table 2. After excluding 14 timeouts, two experts independently annotate all 286 retained trajectories using seven error subtypes (Cohen's $\kappa = 0.81$). Figure 4 pairs the full bench-

mark metrics from Table 2 with failure mechanisms from this sample. The radar axes are independently scaled: the displayed lower and upper bounds define the center and outer polygon for each metric. Appendix D.3 details expert annotation, disagreement resolution, and subtype definitions.

The profiles in Figure 4(a) show contrasting strengths: GPT 5.6 sol -max and GLM 5.3 reject invalid candidates more reliably than they accept valid alternatives, whereas Qwen3.8 Max shows the reverse. Panel (b) summarizes 99 coverage gaps (34.6%), 143 faulty checks (50.0%), and 44 execution logic errors (15.4%). **Coverage gaps.** Tests can exercise an operation while missing the input combinations that expose a defect. A poker suite never compares an ace low straight against three of a kind, allowing a candidate that misclassifies the straight as a high card to pass. **Faulty checks.** The largest group shows that exercising relevant behavior does not ensure that tests judge it correctly, consistent with prior observations of incorrect generated assertions (Yuan et al., 2024). A bracket parser suite includes unmatched parentheses but checks only the output count, accepting a candidate that discards valid groups. Both the poker and bracket suites fail on the initial program and pass on the reference, yet accept an invalid alternative. These paired outcomes show how missing input combinations and weak assertions can survive reference evaluation while additional patches expose them. **Execution logic errors.** Errors in the tests' own execution can also prevent them from evaluating candidate behavior. An EvoEval suite accesses a function stored on its test class through `self`, unintentionally binding an extra argument and failing even on the reference repair. Such failures explain broader reliability problems but do not establish a single reference blind spot.

> **Answer to RQ3: Tests fail through coverage gaps, faulty checks, and execution logic errors.** The analyzed tests omit relevant behavior, misjudge candidate correctness, or fail during execution, helping explain limited reliability. Paired evidence shows that missing input combinations and weak assertions survive reference evaluation but are exposed by additional invalid patches.

4.4 RQ4: Evaluating TestHelix

How do heterogeneous synthesis, peer cross validation, and recursive self improvement contribute to joint test reliability?

Table 3 compares five TestHelix configurations for GPT 5.6 sol and Kimi K3, alongside native agent comparators, Codex (max) and Kimi Code, respectively. Within each model group, the TestHelix configurations share the same base model and tools and use the same total inference cap per benchmark task, shared across pairs. We compare single pair generation with three heterogeneous pairs followed by random submission of one locally qualified suite, then enable peer cross validation at $K = 3$ and $K = 5$. The final configuration applies RSI at $K = 3$ using one shared strategy optimized with Opus 4.6 on 600 training tasks for Reference Acceptance Rate and frozen for evaluation with both models. The other configurations retain the initial strategy. Appendix D.4 provides the experimental controls, and Appendix C.3 details the strategy optimization.

Table 3: **Native agent comparisons and TestHelix ablations.** HS denotes heterogeneous synthesis of K test and repair pairs; PCV denotes peer cross validation. All scores are percentages.

Setting			GPT 5.6 sol					Kimi K3				
HS (K)	PCV	RSI	RAR	VAR	IRR	H	**JSF**	RAR	VAR	IRR	H	**JSF**
Native agent			54.0	61.3	86.0	71.6	23.3	56.6	73.6	**83.0**	**72.0**	64.67
1	✗	✗	51.0	67.5	86.7	69.1	20.3	53.6	76.2	50.9	62.7	41.67
3	✗	✗	51.3	58.0	**87.2**	69.7	21.0	54.0	67.0	71.3	63.3	62.67
3	✓	✗	57.6	64.2	85.3	73.3	27.6	58.0	60.0	86.8	67.4	29.00
5	✓	✗	57.3	65.2	85.7	74.1	28.3	57.3	61.5	86.2	68.0	29.67
3	✓	✓	**60.6**	**69.1**	87.2	**77.1**	**32.3**	**62.6**	**64.2**	78.6	**70.7**	**33.33**

Across both models, larger gains come from using the generated pairs to improve tests than from heterogeneous synthesis without peer cross validation. Generating three heterogeneous pairs and randomly submitting one improves Joint Success Function by at most one percentage point over single pair generation. Enabling the complete peer cross validation process at the same $K = 3$ adds more than six points, supporting the use of alternative repair hypotheses to challenge, refine, and select tests before submission. Expanding peer cross validation to five pairs yields only 0.67 additional points, whereas RSI improves the three pair process by 4.33 to 4.67 points. Under these inference caps, improving how tests are generated and checked therefore offers a larger gain than allocating the budget to additional pairs. The evolved configuration achieves the highest Joint Success Function for both models and exceeds their native coding agents, suggesting that the generation workflow remains a useful target for improving reliability with a fixed base model.

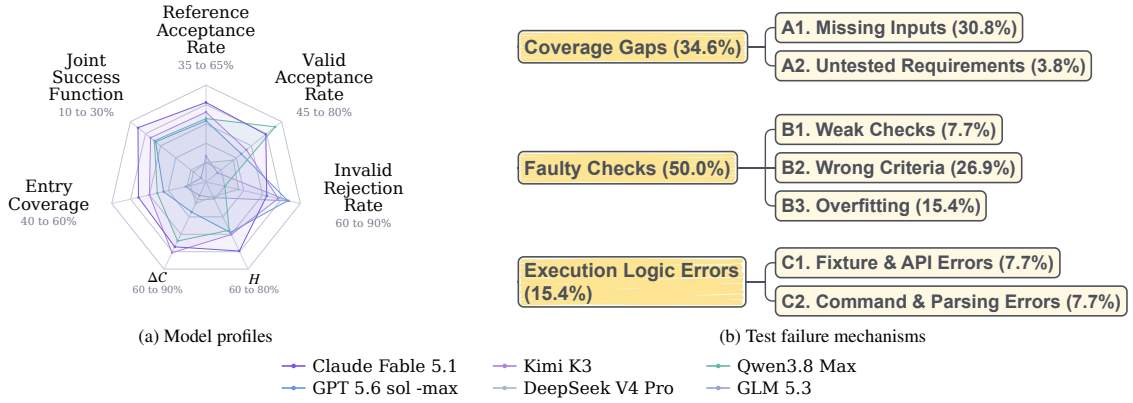


Figure 4: Model profiles across all 300 benchmark tasks (a) and failure mechanisms in 286 sampled failed trajectories after excluding 14 timeouts (b).

> **Answer to RQ4: Heterogeneous synthesis helps tests generalize across implementations.** Peer cross validation uses diverse repairs to check and correct tests, while RSI adapts test generation strategies across tasks. This process improves Joint Success Function over Single by 11.67 to 12.00 percentage points under matched inference budget caps, with gains from each component in the ablations.

5 Related Work

Coding agents and test generation benchmarks. SWE bench evaluates repository changes with executable tests (Jimenez et al., 2024; Yang et al., 2024b); Terminal Bench assesses command line tasks (Merrill et al., 2026), and ContextBench evaluates context retrieval (Li et al., 2026c). SWT Bench studies tests derived from GitHub issues (Mündler et al., 2025); RESTestBench evaluates requirement based REST API tests using related mutants (Kogler et al., 2026). TestPrism evaluates test generation across heterogeneous sources using fixed valid and invalid implementation panels. TestHelix uses AutoSaddler (Park et al., 2026) to search a large semantic space of test generation strategies, drawing on Meta-Harness (Lee et al., 2026) and prior work on reflective feedback and prompt optimization (Shinn et al., 2023; Khattab et al., 2023; Yuksekgonul et al., 2024; Agrawal et al., 2026).

Test generation and coverage. Randoop and EvoSuite use random generation and search (Pacheco and Ernst, 2007; Fraser and Arcuri, 2011). Language model approaches evaluate execution and coverage (Schäfer et al., 2024; Xu et al., 2025b). CodaMOSA and CoverUp use coverage feedback to guide generation (Lemieux et al., 2023; Pizzorno and Berger, 2025), while SymPrompt structures prompts around execution paths (Ryan et al., 2024). TestExplora studies repository test generation and Entry Coverage (Liu et al., 2026). TestPrism reports task scoped coverage alongside behavioral discrimination.

Test adequacy and executable oracles. Mutation testing assesses rejection of faulty variants (Jia and Harman, 2011). PROBE augments repair benchmark tests with reference variants and checks robustness under behavior preserving transformations (Li et al., 2026a). Joint Success Function jointly requires acceptance of valid and rejection of invalid candidates. The oracle problem and patch plausibility limit executable verdicts (Barr et al., 2015; Qi et al., 2015): broader observed behavior does not establish label completeness or semantic correctness. CodeT selects code through agreement with generated tests (Chen et al., 2022); CodeRL combines test feedback with a learned critic to refine code (Le et al., 2022).

6 Conclusion

We introduced TestPrism to evaluate generated tests beyond a single reference. Its Joint Success Function checks acceptance of valid alternatives and rejection of defective implementations across diverse candidate sets. Our evaluation reveals persistent limitations in frontier coding agents; analysis of 286 retained failed trajectories across six model families identifies coverage gaps, faulty checks, and execution logic errors. TestHelix improves joint success through heterogeneous synthesis, peer cross validation, and recursive self improvement, supporting more reliable test generation across implementations.

Limitations

Reliable test generation spans a broad space of task specifications, inputs, implementations, and execution environments. This study focuses on one part of that problem: whether tests written by coding agents distinguish valid and invalid implementations under fixed task contracts. TestPrism evaluates ten labeled implementations per task, and Joint Success Function measures joint success within these finite candidate patch sets. The comparisons concern complete deployed agent configurations, while the TestHelix ablations examine component contributions within each model group. The taxonomy complements these results by characterizing recurring failure mechanisms in 286 analyzed failed trajectories, after excluding 14 timeouts from 300 trajectories sampled equally from the highest Joint Success Function configuration in each of six model families. Failed trajectories have a per-task joint success indicator of zero. The category proportions describe the retained sample and exclude the timeout trajectories.

Source gold tests. We select established public benchmarks with documented test validation, including the widely adopted SWE bench and Terminal Bench families (Appendix B.3.1). Their original quality controls provide the foundation for task and verifier quality. Building on this foundation, our second-stage validation checks adapted verifiers through execution and independent expert review for requirement alignment and potential reference overfitting; tasks with unresolved concerns are excluded. Our scope is evaluating generated tests across implementations under these validated task contracts. Independently re-establishing the correctness of each upstream benchmark is outside this study.

Panel composition and diversity. The prevalence of valid and invalid human patches is difficult to establish and can vary across tasks and development settings. Our equal allocation provides a balanced evaluation design without estimating this prevalence. The diversity contributed by a candidate also depends on task semantics, implementation choices, defect patterns, and its relationship to candidates already in the panel. Consequently, a valid to invalid ratio alone does not determine diversity, and equal counts do not guarantee equal difficulty for acceptance and rejection. Our admission checks use mean AST distance for the valid group and failed-check sets with conditional AST filtering for the invalid group. They do not establish exhaustive semantic coverage or an optimal candidate set. Jaccard distance summarizes overlap in the chosen AST feature representation and can differ from semantic distance; its mean over a panel need not increase with every added patch. This work introduces Joint Success Function and candidate patch sets with explicit diversity controls. Optimizing the label ratio, panel size, or candidate selection is outside its scope.

Extending evaluation to evolving specifications, broader implementation spaces, and additional execution environments remains a direction for future work.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *Preprint*, arXiv:2507.19457.

Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *Preprint*, arXiv:2412.02883.

Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. *Preprint*, arXiv:2402.09171.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *Preprint*, arXiv:2505.20411.

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. *Preprint*, arXiv:2207.10397.

Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. *Preprint*, arXiv:2305.04764.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, and 3 others. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? *Preprint*, arXiv:2509.16941.

Yukang Feng, Jianwen Sun, Zelai Yang, Jiaxin Ai, Chuanhao Li, Zizhen Li, Fanrui Zhang, Kang He, Rui Ma, Jifan Lin, Jie Sun, Yang Xiao, Sizhuo Zhou, Wenxiao Wu, Yiming Liu, Pengfei Liu, Yu Qiao, Shenglin Zhang, and Kaipeng Zhang. 2026. LongCLI-Bench: A Preliminary Benchmark and Study for Long-horizon Agentic Programming in Command-Line Interfaces. *Preprint*, arXiv:2602.14337.

Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, ESEC/FSE'11, pages 416–419. ACM.

Xinyi He, Qian Liu, Mingzhe Du, Lin Yan, Zhijie Fan, Yiming Huang, Yin Zheng, Zejian Yuan, and Zejun Ma. 2026. SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories? *Preprint*, arXiv:2507.12415.

Laura Inozemtseva and Reid Holmes. 2014. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering*, ICSE '14, pages 435–445. ACM.

Kush Jain, Gabriel Synnaeve, and Baptiste Roziere. 2025. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. In *The Thirteenth International Conference on Learning Representations*.

Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5):649–678.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *Preprint*, arXiv:2310.06770.

René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, SIGSOFT/FSE'14, pages 654–665. ACM.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *Preprint*, arXiv:2310.03714.

Leon Kogler, Stefan Hangler, Maximilian Ehrhart, Benedikt Dornauer, Roland Wuersching, and Peter Schrammel. 2026. RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements. *Preprint*, arXiv:2604.25862.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. *Preprint*, arXiv:2207.01780.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-End Optimization of Model Harnesses. *Preprint*, arXiv:2603.28052.

Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE.

Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2024. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study. *Preprint*, arXiv:2403.08604.

Chenglin Li, Yisen Xu, Zehao Wang, Shin Hwei Tan, and Tse-Hsun (Peter) Chen. 2026a. Probe to Generate: Program Variant-Guided Test Augmentation for Repository-Level Repair Benchmarks. *Preprint*, arXiv:2604.01518.

Han Li, Jinyu Tian, Rili Feng, Yuqiao Du, Chong Zheng, Chenyu Wang, Chenchen Liu, Shihao Li, Xinping Lei, Yifan Yao, Weihao Xie, Letian Zhu, and Jiaheng Liu. 2026b. Solvita: Enhancing Large Language Models for Competitive Programming via Agentic Evolution. *Preprint*, arXiv:2605.15301.

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026c. ContextBench: A Benchmark for Context Retrieval in Coding Agents. *Preprint*, arXiv:2602.05892.

Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: a multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, SPLASH '17, pages 55–56. ACM.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *Preprint*, arXiv:2305.01210.

Steven Liu, Jane Luo, Xin Zhang, Aofan Liu, Hao Liu, Jie Wu, Ziyang Huang, Yangyu Huang, Yu Kang, and Scarlett Li. 2026. TestExplora: Benchmarking LLMs for Proactive Bug Discovery via Repository-Level Test Generation. *Preprint*, arXiv:2602.10471.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, and 66 others. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. *Preprint*, arXiv:2601.11868.

Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2025. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. *Preprint*, arXiv:2406.12952.

Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, OOPSLA07, pages 815–816. ACM.

Yue Pan, Zimin Chen, Siyu Lu, Zhaoyang Chu, Xiang Li, Han Li, Yang Feng, Claire Le Goues, Federica Sarro, Martin Monperrus, and He Ye. 2026. Prometheus: Towards Long-Horizon Codebase Navigation for Repository-Level Problem Solving. *Preprint*, arXiv:2507.19942.

Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2026. AutoSaddler: Automatic Harness Optimization with Durable Updates from Agent Execution Traces. *Preprint*, arXiv:2608.23041.

Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Preprint*, arXiv:2403.16218.

Ori Press, Brandon Amos, Haoyu Zhao, Yikai Wu, Samuel K. Ainsworth, Dominik Krupke, Patrick Kidger, Touqir Sajed, Bartolomeo Stellato, Jisun Park, Nathanael Bosch, Eli Meril, Albert Steppi, Arman Zharmagambetov, Fangzhao Zhang, David Perez-Pineiro, Alberto Mercurio, Ni Zhan, Talor Abramovich, and 5 others. 2025. AlgoTune: Can Language Models Speed Up General-Purpose Numerical Programs? *Preprint*, arXiv:2507.15887.

Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ISSTA '15, pages 24–36. ACM.

Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, Anoop Deoras, Giovanni Zappella, and Laurent Callot. 2025. SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents. *Preprint*, arXiv:2504.08703.

Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proceedings of the ACM on Software Engineering*, 1(FSE):951–971.

Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering*, 50(1):85–105.

Quan Shi, Michael Tang, Karthik Narasimhan, and Shunyu Yao. 2024. Can Language Models Solve Olympiad Programming? *Preprint*, arXiv:2404.10952.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *Preprint*, arXiv:2303.11366.

Atharv Sonwane, Eng-Shen Tu, Wei-Chung Lu, Claas Beger, Carter Larsen, Debjit Dhar, Simon Alford, Rachel Chen, Ronit Pattanayak, Tuan Anh Dang, Guohao Chen, Gloria Geng, Kevin Ellis, and Saikat Dutta. 2026. OmniCode: A benchmark for evaluating software development agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 40634–40661, San Diego, California, United States. Association for Computational Linguistics.

Yuxuan Sun, Yuze Zhao, Yufeng Wang, Yao Du, Zhiyuan Ma, Jinbo Wang, Mengdi Zhang, Kai Zhang, and Zhenya Huang. 2026. SWE-Mutation: Can LLMs Generate Reliable Test Suites in Software Engineering? *Preprint*, arXiv:2605.22175.

Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, Yang Li, Zhongwen Li, Shirong Lin, Jiashun Liu, Zenan Liu, Tao Luo, Dilxat Muhtar, Yuanbin Qu, Jiaqiang Shi, and 71 others. 2026a. Let It Flow: Agentic Crafting on Rock and Roll, Building the ROME Model within an Open Agentic Learning Ecosystem. *Preprint*, arXiv:2512.24873.

Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025a. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. *Preprint*, arXiv:2406.04531.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. *Preprint*, arXiv:2402.01030.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025b. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *Preprint*, arXiv:2407.16741.

Yibo Wang, Congying Xia, Wenting Zhao, Jiangshu Du, Chunyu Miao, Zhongfen Deng, Philip S. Yu, and Chen Xing. 2026b. MultiFileTest: A multi-file-level LLM unit test generation benchmark and impact of error fixing mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 28151–28172, San Diego, California, United States. Association for Computational Linguistics.

Chunqiu Steven Xia, Yinlin Deng, and Lingming Zhang. 2024. Top Leaderboard Ranking = Top Coding Proficiency, Always? EvoEval: Evolving Coding Benchmarks via LLM. *Preprint*, arXiv:2403.19114.

Jiacheng Xu, Bo Pang, Jin Qu, Hiroaki Hayashi, Caiming Xiong, and Yingbo Zhou. 2025a. CLOVER: A Test Case Generation Benchmark with Coverage, Long-Context, and Verification. *Preprint*, arXiv:2502.08806.

Jingxuan Xu, Ken Deng, Weihao Li, Songwei Yu, Huaixi Tang, Haoyang Huang, Zhiyi Lai, Zizheng Zhan, Yanan Wu, Chenchen Zhang, Kepeng Lei, Yifan Yao, Xinping Lei, Wenqiang Zhu, Zongxian Feng, Han Li, Junqi Xiong, Dailin Li, Zuchen Gao, and 20 others. 2025b. SWE-Compass: Towards Unified Evaluation of Agentic Coding Abilities for Large Language Models. *Preprint*, arXiv:2511.05459.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024a. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Preprint*, arXiv:2405.15793.

John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. 2024b. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains? *Preprint*, arXiv:2410.03859.

Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, pages 283–294. ACM.

Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2024. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *Preprint*, arXiv:2305.04207.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "Differentiation" via Text. *Preprint*, arXiv:2406.07496.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. SWE-bench Goes Live! *Preprint*, arXiv:2505.23419.

Hong Zhu, Patrick A. V. Hall, and John H. R. May. 1997. Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4):366–427.

AI use statement

LLM based coding agents generate candidate implementations and tests (Sections 2.3 and 3). Generative AI tools, including Codex, assisted with the evaluation framework, metric terminology, research questions, TestHelix comparisons and ablation design, data processing and visualization code, inventory summaries, literature searches, and manuscript drafting, organization, and editing. Source verifiers determine candidate labels; inventory counts are checked against frozen manifests. Experts annotated the sampled test failure trajectories (Appendix D.3). Manuscript and figure updates were checked through compilation, reference checks, and visual inspection. The authors take responsibility for the final text, claims, code, and artifacts, including AI assisted material.

Ethics statement

TestPrism evaluates coding agents on software requirements and candidate implementations. Appendix B.5 and Section 2.3 record source and candidate provenance and verifier labels. Reuse and redistribution remain subject to applicable licenses and attribution requirements. Generated programs and tests are untrusted code requiring restricted access to host resources, credentials, and external services. Performance on a finite panel does not certify software correctness or security. Appendix 6 discusses source validity, finite panel scope, and comparison limits.

Reproducibility statement

Section 2 describes benchmark construction; Appendix B follows its construction stages, candidate patch set, and benchmark statistics. Section 2.6 and Appendix B.6 define metrics, denominators, coverage collection, and missing observations. Section 3 describes TestHelix; Appendix C follows its synthesis, cross validation, and self improvement stages, including the strategy schema, prompt excerpts, AutoSaddler configuration, and independent validation results. The final frozen strategy is provided in the accompanying TestHelix source code. The training task manifest is supplied in the supporting materials. Appendix D follows RQ1 through RQ4, covering baseline configurations, patch diversity, failure annotation, cumulative controls, submission rules, and resource accounting. Tables 2, 14, and 3 report the system and component comparisons. Reproduction requires the same task and candidate patch sets, model configurations, inference limits, and execution environments; task level records are needed to recompute outcomes and resource use.

Appendix Guide

The appendices follow the order of the main text. Each entry below links to the corresponding supporting material.

Main text	Appendix	Page
Introduction	A. Benchmark comparison	13
TestPrism	B. Construction and evaluation	15
TestHelix	C. Method and strategy details	22
Experiments	D. Results and analysis	27

Research questions: RQ1 (D.1) RQ2 (D.2) RQ3 (D.3) RQ4 (D.4).

A Benchmark Comparison Conventions

Table 1 compares behavioral and coverage targets: a check marks the corresponding target, a dagger marks a related check with different granularity or definitions, and a cross marks its absence from the main protocol. Reference Acceptance Rate measures the fail to pass transition from the initial program to the reference implementation.

Behavioral and coverage targets. TestGenEval uses mutation testing; CLOVER and MultiFileTest evaluate tests through execution and code coverage (Jain et al., 2025; Xu et al., 2025a; Wang et al., 2026b). Their file, block, or project coverage is distinct from reference change and entry scope coverage. SWT Bench and TDD Bench Verified measure changed code coverage (Mündler et al., 2025; Ahmed et al., 2024). OmniCode jointly requires reference acceptance and rejection of all bad patches, while SWE Mutation reports mutant detection (Sonwane et al., 2026; Sun et al., 2026). These checks provide evidence beyond one reference; acceptance of valid alternatives under Joint Success Function is outside their scope.

Counting conventions. Variants counts implementations other than the reference, including mutants; a check indicates that variants are used but no total is established. SWT Bench's separate patch filtering experiment is outside that column's main protocol scope. TestPrism has 2,700 such variants and 300 reference solutions. Languages counts individual task implementation languages. The 300 TestPrism tasks use Python (205), TypeScript (44),

Java (19), JavaScript (16), C++ (8), and Go (8). Scope identifies test granularity; TestPrism spans local function and file tests, multifile projects, and repository issues.

B TestPrism: Construction and Evaluation

This appendix follows the construction steps, benchmark statistics, and evaluation protocol in Section 2.

B.1 Task Screening and Adaptation

Source selection and prior evidence. We select public benchmarks with traceable task origins and documented test validation. The SWE bench and Terminal Bench families maintain public leaderboards with results from multiple agent developers. Other sources draw on community maintained Exercism exercises, official USACO competitions, and published benchmarks for specialized programming settings.

Task adaptation. We retain tasks centered on code and suitable for executable test authoring, excluding tasks with unusable Docker images or environments that cannot be reproduced. Adapters map tasks from all 17 source benchmarks to Harbor's instruction, environment, tests, and solution components. Dependencies, services, and execution limits follow each source so that its original functional and performance requirements can be checked.

B.2 Task Selection

Test set selection and eligibility. Initial screening excludes tasks outside the code testing scope and tasks whose environments cannot be reproduced. The selection signals in Section 2.2 use effective lines in `solution.sh`, excluding blank and comment lines (`solution_lines`); test functions enumerated from source files or benchmark test metadata (`test_func_count`); and this count divided by the number of detected modified files, with a minimum denominator of one (`test_coverage_ratio`). The ratio describes test density. We apply source-specific lower bounds calibrated to each source's distribution. Sources with smaller typical solutions or lower enumerated test counts use correspondingly lower bounds, preserving their representation alongside sources with larger values. This accounts for differences in task granularity and verifier structure; source quotas additionally balance the benchmark composition. After enrichment, final admission requires applicability annotations for both $\Delta\mathcal{C}$ and Entry Coverage, ten physical candidate artifacts meeting the diversity criteria in Appendix B.4, and agreement between each artifact digest and its manifest.

Table 4 lists the source-specific thresholds applied during difficulty screening. Source test counts follow the complete and partial enumeration convention in Appendix B.5. Density uses original patch paths for 204 tasks and reference file changes reconstructed from initial task files and build specifications for the remaining 96. It counts distinct changed, added, or removed files, including code, configuration, tests, and documentation; unchanged rewrites, temporary helpers, and runtime or build outputs are excluded.

Table 4: **Source-specific filtering thresholds.** Task counts describe the final test set.

Source	Tasks	Lines	Tests	Density
Aider Polyglot	8	32	20	20
AlgoTune	2	73	2	0.5
DevEval	12	7	2	0.2
EvoEval	20	29	2	2
Long CLI	1	411	28	28/3
QuixBugs	16	36	2	2
SWE bench	33	11	9	13/3
SWE bench Live	28	14	7,424	3871/3
SWE bench Multimodal	11	21	30	2
SWE Perf	21	32	3	0.6
SWE rebench	46	17	16	4/15
SWE bench Pro	17	669	1	1/15
SWE PolyBench	48	18	11	1.375
Terminal Bench	8	83	4	4
Terminal Bench 2	4	15	6	6
Terminal Bench Pro	5	66	12	6
USACO	20	9	2	2

Training and test split. Tasks are normalized across sources and deduplicated before partitioning. Difficulty screening operates within source benchmarks. Source quotas form a training set of 1,700 tasks and a disjoint candidate test pool. The training set is fixed at this stage; candidate test tasks continue through verifier validation, patch generation, and diversity filtering. We select the final 300 eligible test tasks with proportional source quotas, retaining every source, and seed 42 for sampling within each source. Tasks not selected for the final test set are excluded from its statistics. Candidate test tasks undergo the verifier checks and independent human review described in Appendix B.3.1 before final admission.

Selection uses task properties and artifact availability, independently of Reference Acceptance Rate, Joint Success Function, or coverage collection success. Test set membership remains fixed when runtime observations are missing.

Filtering funnel. Table 5 summarizes construc-

tion from an initial pool of 15,148 source tasks. The final dataset contains 1,700 training tasks and 300 test tasks. All entries count tasks, including stages that assess their patch sets. Source quota exclusions denote tasks left unselected, rather than failures of the quality checks. After the split, the training set remains fixed and only the candidate test pool continues through verifier validation, initial panel assembly, and diversity refinement. Initial assembly requires one reference, four valid alternatives, and five invalid implementations. Tasks that exhaust the shared generation budget before filling these quotas fail initial assembly. Tasks that fill the quotas but still fail either group criterion when the same budget is exhausted are excluded during diversity refinement (Appendix B.4).

Table 5: **Task filtering stages in TestPrism construction.**

Stage	Retained	Excluded	Rate
Shared screening			
Code focus and test authoring suitability	12,937	2,211	14.6%
Docker and environment reproducibility	10,984	1,953	15.1%
Solution size, test count, and test density	6,731	4,253	38.7%
Source quota selection	2,473	4,258	63.3%
Split: 1,700 training tasks and 773 candidate test tasks			
Candidate test pool only			
Verifier checks and human review	608	165	21.3%
Initial candidate panel assembly	394	214	35.2%
Within-group diversity refinement	311	83	21.1%
Final source quota selection	300	11	3.5%

B.3 Patch Generation and Validation

B.3.1 Source Test Validation

Table 6 links all 17 sources to their test provenance and original quality controls, including execution checks, expert review, and validation against reference or alternative solutions.

Additional validation after adaptation. The source benchmarks provide the original validation; our additional validation round checks the adapted verifiers before candidate generation. We preserve the source verifiers during adaptation and require them to reject the initial state and accept the reference solution before using them to label candidates. For performance tasks, failure means the initial, unaccelerated program misses the task's performance requirement; passing requires an accelerated solution that meets the original task-defined thresholds while preserving functional correctness. Candidate labels are established independently of the generated tests under evaluation and remain fixed across agents.

Independent verifier review. Before candidate generation, two experts independently review the task requirements and source verifier, judging whether the available information supports test authoring. Drawing on development experience, they assess potential overfitting to the reference and gaps in requirement coverage, including checks that could reject valid alternatives or miss incorrect behavior. Disagreements are discussed to clarify the concerns; any task with a remaining concern about verifier usability is excluded. Agreement on the independent usability judgments, before adjudication, is Cohen's $\kappa = 0.68$. This review complements the source validation procedures and automatic execution checks; retained candidate labels follow the verifier outcomes defined in Section 2.3.

B.3.2 Candidate Generation and Labeling

Only the candidate test pool undergoes patch generation, using different coding agents and models of varying capabilities to produce alternative implementations. These runs capture multiple ways of satisfying the task requirements and errors arising during task solving. Each candidate is stored as a patch artifact. The admission gate removes empty submissions and exact duplicates, then checks applicability in the adapted task environment. The source verifier executes its checks in a clean environment, producing a matrix of candidate outcomes. A candidate is valid when every check passes and invalid when any check fails, regardless of its intended correctness. We retain its provenance, identifier, label, and digest for selection and evaluation. Each task has a single budget of 30 generated patch outputs, shared by initial panel assembly and diversity refinement. The existing reference does not consume this budget. Every produced patch counts once, including patches later rejected or not selected. No per-label generation cap or separate refinement-round cap applies.

B.3.3 Candidate Patch Set

Table 7 summarizes all 3000 candidate artifacts belonging to the fixed task manifest. Each task contributes one reference implementation, four other valid implementations, and five invalid implemen-

Table 6: **Source test provenance and prior validation evidence.**

Source	Test provenance	Original quality controls
Aider Polyglot	Community maintained Exercism suites	Expected outputs, example implementations, and reviewed canonical specifications support specification checks and traceability.
AlgoTune	Expert solvers, input generators, and verifiers	Two additional contributors review each task; outputs from different solver seeds check acceptance of alternative solutions (Press et al., 2025).
DevEval (DevBench)	Project unit and acceptance tests	Annotators execute reference code and supplement existing tests to improve coverage (Li et al., 2024).
EvoEval	Generated tasks, solutions, and augmented tests	Authors inspect tasks and reference solutions; iterative solution agreement resolves ambiguities (Xia et al., 2024).
Long CLI	Tests written from requirements alone	Tests are authored without the solution repository; initial and solution states undergo failure and regression checks followed by expert review (Feng et al., 2026).
QuixBugs	Quixey challenge programs with passing and failing cases	Released drivers compare buggy and corrected programs on supplied cases (Lin et al., 2017).
SWE bench	Developer tests from merged pull requests	Execution requires a fail to pass transition; installation and runtime failures are filtered (Jimenez et al., 2024).
SWE bench Live	Tests from issue resolving pull requests	Validation requires a fail to pass transition and consistent results across repeated runs (Zhang et al., 2025).
SWE bench Multimodal	Repository tests for visual software issues	Ten validation runs remove inconsistent tests; authors inspect every task for feasibility (Yang et al., 2024b).
SWE Perf	Repository tests for performance changes	Selected tests pass before and after optimization; timing validation uses 20 repetitions and statistical filtering (He et al., 2026).
SWE rebench	Test patches from resolving pull requests	Reference patches must fix initial test failures while preserving initially passing tests (Badertdinov et al., 2025).
SWE bench Pro	Repository tests with human review	Repeated runs check stability; human reviewers remove irrelevant or overly broad tests (Deng et al., 2025).
SWE PolyBench	Repository tests and pull request updates	Manually configured containers check failure transitions and regression preservation, excluding untestable changes (Rashid et al., 2025).
Terminal Bench	Contributor tests and reference solutions	Contribution checks cover specification alignment, reference success, and failure without agent actions.
Terminal Bench 2	Contributor tests and manual reference solutions	Three reviewers check specification, execution, and exploits, averaging three review hours per task (Merrill et al., 2026).
Terminal Bench Pro	Programmer authored instructions and unit tests	Independent experts review ambiguity, false positives, reference solutions, and coverage over multiple rounds (Wang et al., 2026a).
USACO	Official hidden tests and editorial based reference code	Reference solutions pass hidden tests; manual resource checks compare correct and incorrect asymptotic complexity (Shi et al., 2024).

tations. Their executable labels and digests are retained together. Every referenced file is present and matches its manifest digest. Candidate labels follow the validated source verifiers described in Appendix B.3.1. Panel diversity is assessed using the criteria for valid and invalid groups in Appendix B.4.

Table 7: **Candidate patch inventory for the fixed TestPrism test set.**

Tasks	Reference	Other valid	Invalid	Verified files	Missing
300	300	1200	1500	3000	0

B.4 Diversity Filtering and Task Conversion

AST distance. We apply each patch to the same initial program and parse the modified production code before and after the change with fixed versions of Tree-sitter and the corresponding language grammars. From AST nodes in the modified regions on each side, we extract ordered local subtrees of depth three, counting the root as depth zero. Features preserve node types, child order, operators, and literal types and values. Identifiers are normalized to a common token; comments, formatting, and source positions do not contribute features.

Let B_p and C_p be the resulting feature sets in the initial and candidate states. We represent the patch by its added and removed structural features:

$$\mathcal{A}(p) = \big(\{+\} \times (C_p \setminus B_p)\big) \cup \big(\{-\} \times (B_p \setminus C_p)\big).$$

Each set depends only on the patch and the common initial program, so its definition remains fixed across comparisons. Features are deduplicated; repeated occurrences do not increase their weight. We use Jaccard distance:

$$d_{\mathrm{AST}}(p, q) = 1 - \frac{|\mathcal{A}(p) \cap \mathcal{A}(q)|}{|\mathcal{A}(p) \cup \mathcal{A}(q)|}.$$

When both sets are empty, the distance is zero; when only one is empty, it is one. Unresolved parse errors in the changed regions leave the distance unavailable. Such a generated candidate cannot enter the selected panel and still counts toward the shared 30-patch generation budget. This measure captures local structural differences; verifier outcomes determine behavioral labels.

Group admission criteria. We first assemble a valid group $V = \{p_{\mathrm{ref}}, v_1, v_2, v_3, v_4\}$ and an invalid group $I = \{i_1, i_2, i_3, i_4, i_5\}$. Each group contains ten distinct unordered pairs. The valid group must satisfy

$$D(V) = \frac{1}{10} \sum_{\{p,q\} \subset V} d_{\mathrm{AST}}(p, q) > 0.5.$$

This mean includes comparisons with the reference and between alternatives; it is not a lower bound on every pair.

For invalid implementations, we first compare the sets of verifier checks they fail. Different failed-check sets satisfy the diversity check directly. Candidates with the same failed-check set must additionally pass the AST Jaccard check. Thus, AST filtering within the invalid group is conditional on matching failed-check sets. We impose no cross-group distance constraint: verifier outcomes already distinguish implementations that pass all checks from those that fail at least one.

Generation and replacement. If either group fails its criterion, we continue generating candidates. Each new candidate undergoes the same applicability, deduplication, and verifier checks. Its observed label determines the group it can enter. When the valid group has not met its criterion, we evaluate each replacement of a nonreference member and choose the one that maximizes $D(V)$, accepting it only if the mean strictly increases. For the invalid group, replacements are assessed by their failed-check sets, with the AST Jaccard check applied when those sets match. The reference remains fixed, and the panel always contains one reference, four valid alternatives, and five invalid implementations. We stop as soon as both criteria hold or after evaluating the thirtieth generated patch. At that limit, tasks without a complete panel satisfying both criteria are excluded. Final source quotas select from the eligible tasks; tasks left unselected at this stage have passed the quality filters. Every retained artifact must match its recorded digest.

Test generation adapter. Conversion primarily updates `task.yaml` to specify submission requirements and `Dockerfile` to prepare the execution environment. The original requirement remains the behavioral contract, and candidate patches are reserved for evaluation. Submitted tests, fixtures, and the executable entry script reside in the designated artifact directory.

Interpretation boundaries. The source and difficulty composition reflects the development collection. Requiring meaningful local production targets

for $\Delta\mathcal{C}$ and Entry Coverage excludes tasks whose essential outcomes lack such targets. Candidate labels follow the validated source task contracts, while panel diversity follows the criteria for valid and invalid groups above. Runtime and coverage observation counts are reported separately in Appendix B.6.

B.5 Benchmark Statistics

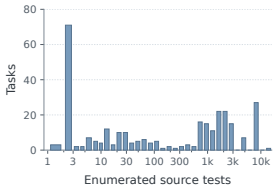


Figure 5: **Distribution of source test counts.**

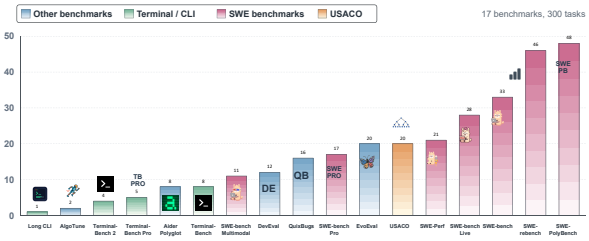


Figure 6: **Test tasks by source benchmark.**

Reading the statistics. Figure 6 reports task counts by source benchmark; Figure 7 reports category and subcategory counts. Figure 5 shows the distribution of enumerated source test counts. Table 9 gives test set task, repository, source test, and patch counts by language, with counting conventions below. Unselected tasks in the candidate test pool do not enter these statistics.

Table 8 and Figure 6 report the task sources in the test set. The counts cover all 300 selected tasks, with all 17 sources retained. Source names identify provenance; each task's requirement and coverage applicability annotations determine its measurement scope. Figure 7 shows the six task categories, 34 subcategories, and their test task counts.

Table 8: **Test task counts by source benchmark.**

Source	Aider Polyglot	AlgoTune	DevEval	EvoEval	Long CLI	QuixBugs
Tasks	8	2	12	20	1	16
Source	SWE bench	SWE bench Live	SWE bench Multimodal	SWE Perf	SWE rebench	SWE bench Pro
Tasks	33	28	11	21	46	17
Source	SWE PolyBench	Terminal Bench	Terminal Bench 2	Terminal Bench Pro	USACO	
Tasks	48	8	4	5	20	



Figure 7: **Task taxonomy of TestPrism.** Numbers indicate task counts; sector angles and areas are not proportional to these counts.

Language and source test counts. Table 9 reports test set task, repository, source test, and patch counts by language. Repository counts cover distinct source repository and project identities recorded in task files, with totals deduplicated

across languages. Source test counts enumerate test identifiers and verifier checks from those files, summed across tasks. When inner cases are enumerated, their aggregate wrapper is not counted again. Test patch counts come from the fixed candidate inventory and include reference solutions, for a total of 3,000.

Table 9: **TestPrism test set statistics by language.** #Test counts enumerated source tests and verifier checks, including partial counts.

Language	#Repo	#Task	#Test	#Patch
Python	46	205	327,627	2,050
JS/TS	9	60	110,954	600
Java	4	19	38	190
C/C++	5	8	50	80
Go	3	8	144	80
Total	67	300	438,813	3,000

Figure 5 uses the same task input inventory: 184 tasks have complete test lists and 116 have partial counts from enumerable verifier functions or performance checks. All 300 tasks have positive enumerable counts; unenumerated internal tests are excluded.

Language labels follow test entry annotations. JS/TS combines JavaScript and TypeScript, and C/C++ combines C and C++.

B.6 Evaluation Protocol

This section expands the auxiliary definitions and collection conventions for the evaluation protocol in Section 2.6.

B.6.1 Behavioral Metric Definitions

For task i, let $o(T_i, p)$ denote a recorded outcome in $\{\text{pass}, \text{fail}, \text{error}\}$ when the submitted entry script T_i runs on implementation p. The acceptance and rejection indicators are

$$a_i(p) = \mathbf{1}\big[o(T_i, p) = \text{pass}\big],$$
$$r_i(p) = \mathbf{1}\big[o(T_i, p) = \text{fail}\big].$$

Both indicators are zero when the outcome is missing or a recognized execution error. The reference transition and full panel success indicators are

$$F_i = r_i(b_i)\, a_i(g_i),$$
$$S_i = F_i \prod_{p \in \mathcal{V}_i \setminus \{g_i\}} a_i(p) \prod_{p \in \mathcal{I}_i} r_i(p).$$

Since $g_i \in \mathcal{V}_i$, substituting F_i into S_i gives the joint criterion in the main text. The benchmark rates average these indicators over the fixed set of $N = 300$ tasks:

$$\text{RAR} = \frac{1}{N}\sum_{i=1}^{N} F_i, \qquad \text{JSF} = \frac{1}{N}\sum_{i=1}^{N} S_i.$$

Let $\mathcal{V}_i^{\text{obs}}$ contain the other valid candidates with recorded outcomes, excluding g_i, and $\mathcal{I}_i^{\text{obs}}$ the invalid candidates with recorded outcomes. The corresponding task rates are

$$A_i = \frac{\sum_{p \in \mathcal{V}_i^{\text{obs}}} a_i(p)}{|\mathcal{V}_i^{\text{obs}}|}, \qquad R_i = \frac{\sum_{p \in \mathcal{I}_i^{\text{obs}}} r_i(p)}{|\mathcal{I}_i^{\text{obs}}|}.$$

Complete observations give denominators of four for A_i and five for R_i; an empty observed denominator leaves the corresponding task rate undefined. Let $\mathcal{T}_V = \{i : |\mathcal{V}_i^{\text{obs}}| > 0\}$ and $\mathcal{T}_I = \{i : |\mathcal{I}_i^{\text{obs}}| > 0\}$. The reported component rates average the defined task rates with equal weight:

$$\text{VAR} = \frac{1}{|\mathcal{T}_V|}\sum_{i \in \mathcal{T}_V} A_i,$$
$$\text{IRR} = \frac{1}{|\mathcal{T}_I|}\sum_{i \in \mathcal{T}_I} R_i.$$

Neither rate requires a successful initial-to-reference transition. Candidates without an execution record are excluded from the corresponding task denominator. Recorded errors remain in it even if execution did not complete, but contribute to neither numerator. A task with an undefined rate is excluded from that metric's mean; an empty task set leaves the aggregate undefined. With complete observations, both task sets contain all N tasks. When every included task has the same candidate count, the task mean equals the ratio of total matching outcomes to total candidate observations. Different task denominators can make these two averages differ.

Every reported baseline and TestHelix configuration has complete observations on all 300 tasks: 1,200 valid-alternative and 1,500 invalid-candidate records.

The harmonic mean H combines the aggregate acceptance and rejection rates:

$$H = \frac{2\,\text{VAR}\,\text{IRR}}{\text{VAR} + \text{IRR}}.$$

It is zero when both rates are zero and undefined when either rate is undefined. We first round Valid Acceptance Rate and Invalid Rejection Rate to the

two decimal places shown as percentages in the tables, compute their harmonic mean, and then round H to two decimal places.

The harness recognizes errors such as a missing entry script or a timeout and otherwise uses the test runner's exit status. A nonzero exit may reflect an assertion failure or another problem inside the script, so interpreting a candidate disagreement requires inspecting the test and source. This limitation also applies to a recorded failure on the initial state.

B.6.2 Coverage Collection and Aggregation

We use Change Coverage from SWT Bench and Entry Coverage from TestExplora (Mündler et al., 2025; Liu et al., 2026), with measurement defined by the execution sets below.

For $\Delta\mathcal{C}$, C_i contains source coordinates changed by the reference solution and executed by trusted tests in the corresponding initial or reference state. Coordinates include the program state: a replaced line can contribute one target in each state. Generated test execution in the corresponding states forms H_i. Trusted test execution fixes the target denominator $|C_i|$. Changed lines without an observed executable event do not contribute to it. The metric does not separately remove formatting edits on executed code lines.

For Entry Coverage, trusted tests identify production entry points with a unique, auditable mapping to local source spans in the reference state. The collector forms E_i from executable coordinates in those spans, using the union of executed and unexecuted coordinates in the generated tests' reference state coverage report. Generated test execution on the reference state forms H_i^g. Trusted tests fix the entry scope; its executable coordinates need not have been executed by trusted tests. Overlapping entry spans count each coordinate once, and the scope can extend beyond changed lines.

The task coverage ratios are

$$\Delta\mathcal{C}_i = \frac{|H_i \cap C_i|}{|C_i|}, \qquad \mathrm{EC}_i = \frac{|H_i^g \cap E_i|}{|E_i|}.$$

Every benchmark task meets both applicability rules. An empty measured denominator or failed collector receives an explicit status and no numeric coverage value. With valid observations for all N tasks, the benchmark coverages are task means:

$$\Delta\mathcal{C} = \frac{1}{N}\sum_{i=1}^{N} \Delta\mathcal{C}_i, \qquad \mathrm{EC} = \frac{1}{N}\sum_{i=1}^{N} \mathrm{EC}_i.$$

All fourteen baseline configurations in Tables 2 and 14 have 300 valid observations for each coverage metric, with no failed collections or empty measured denominators.

B.6.3 Mapping Stored Metrics to Reported Quantities

The stored `f2p_rate` field records the single reference success indicator F_i: initial failure and reference acceptance by the submitted entry script. Reference Acceptance Rate averages this indicator over the fixed benchmark, with missing or erroneous required outcomes receiving no success credit. A pass on the reference alone remains an execution record and is not a separate reported metric.

The stored valid acceptance rate includes all five valid candidates, including the reference. The task rate A_i above covers the other four candidates for Valid Acceptance Rate. Invalid Rejection Rate measures rejection of the invalid candidates. Both metrics first compute each task's rate from recorded outcomes, including recorded errors as unmatched, and then average the defined task rates with equal weight. This follows the task averaging convention of the stored run summaries. The stored valid acceptance rate retains the reference, so its task rate must be recomputed over the other four valid candidates before averaging to report Valid Acceptance Rate. The stored balanced patch accuracy is the arithmetic mean of the stored valid acceptance rate and invalid rejection rate. The mutation score aliases the stored invalid rejection rate, so it provides no additional independent observation.

Joint Success Function averages the stored task resolution indicator over the fixed benchmark. A positive indicator requires initial failure, acceptance of every valid candidate, rejection of every invalid candidate, and no recorded execution error. Missing required outcomes receive no joint success credit.

Change Coverage ($\Delta\mathcal{C}$) is stored in `gold_patch_coverage`; the general coverage field aliases its ratio. Entry Coverage is stored in `entry_coverage`. Missing or undefined coverage values remain observation gaps rather than numeric zeros. These correspondences are checked alongside task membership and panel identity in the analysis artifact.

C TestHelix: Method and Strategy Details

This appendix follows heterogeneous synthesis, peer cross validation, and recursive self improvement in Section 3. The component comparisons appear in Appendix D.4.

This appendix documents the TestHelix strategy schema and fixed prompt templates, followed by the RSI search configuration and independent validation protocol.

C.1 Heterogeneous Synthesis

Prompt Templates. The following passages preserve the fixed prompt wording, including executable names and punctuation; separate quote blocks are excerpts rather than one continuous prompt. Task text, branch strategy, guidance, retrieved lessons, execution observations, and remaining budget are inserted at runtime. In these prompts, `policy` refers to the configurable test generation strategy.

Candidate generation. The candidate-generation template contains the following instructions.

01 Candidate generation PROMPT EXCERPTS

```
The original public task is the
acceptance contract.
```

```
The unchanged baseline must fail for
a task-related reason.  Derive
expectations independently from
public evidence, not from the repair.
Neither a command exit code nor a
host checkpoint proves correctness.
```

```
Only agent-tests/ at the original
working directory is exported; keep
that location unchanged even if
public source lives elsewhere.  Keep
test scripts, fixtures and helpers
there, with executable
agent-tests/run-tests.sh.  Installed
environments, builds and caches
belong outside the export; the
runner must reproduce required
setup.
```

```
The host independently executes and
reviews the candidate; your
declaration cannot bypass
verification.  Use only public task
evidence and task-provided source;
hidden benchmark files, reference
solutions and other candidates are
unavailable.
```

01 Candidate generation CONTINUED

```
Choose the next action from the
missing evidence and remaining
budget.  Skills provide optional
methods, not additional acceptance
requirements.  Batch independent
reads, follow artifact references
and keep concise evidence in
update_notes.  Never emit
submit_candidate in the same
response as writes or exec:  write
the source/replay first, execute the
exact runner in a later response,
then submit alone.
```

With the shared `execution-positive` ablation fallback, `candidateWorkflow` replaces the optional repair sentence with:

02 Publication fallback PROMPT EXCERPTS

```
The shared publication fallback
requires a captured source repair
that passes your tests in a fresh
sandbox, together with a
reproducible baseline failure.
Preserve and submit incomplete
artifacts honestly if you cannot
establish both controls.
```

Branch strategies. At $K = 3$, the three synthesis branches use the following exploration strategies. Each strategy changes exploration focus while retaining the original public acceptance contract.

03 Branch strategies PROMPT EXCERPTS

Branch 1

```
Write a small public-behavior
regression early, then develop a
minimal source repair and expand the
suite with an independent
small-input oracle where possible.
```

Branch 2

```
Independently solve the complete
task.  Derive a small oracle or
boundary table from public examples,
mathematics or documented behavior
before implementing the repair.
Check one ordinary supported case as
well as the regression.  Choose the
repair independently of the test
oracle.  Inspect uncertain expected
values against the original contract;
do not inherit another solver's plan
or assumptions.
```

Branch 3

```
Challenge likely incomplete
implementations.  Cover explicit
requirements overlooked by narrow
tests and avoid adding rules the
task does not state.
```

Direction planning. The direction-planning prompt guides the first solver; peers retain independent strategies. The prompt includes the following instructions.

04 Direction planning PROMPT EXCERPTS

```
Plan a complete
regression-and-repair workflow from
the original public contract and
observed environment.  Return one
branch for the first solver:  write
and run public-contract tests,
implement a minimal source repair as
an internal positive control, rerun,
and submit.  Completion requires
baseline failure and repaired-source
success; only tests are exported.
```

```
Independent peers derive their own
approach without this plan.  Cite
each requirement by taskLines in the
numbered original task; the host
extracts quotes.  Do not invent
expected values, implementations,
tooling availability, or new
acceptance rules.  Prefer one
focused inspection that resolves a
concrete unknown over a broad
repository survey.  The host owns
budgets and independent validation.
Replanning must address the observed
invalid premise, preserving
unaffected work.
```

C.2 Peer Cross Validation

Evidence boundary. Generation uses the original public task and public source. Internal repairs are hypotheses used as execution controls. Review runs in a fresh copy of the original source with the generated tests; no generated source repair is installed there. The host supplies bounded execution symptoms and checks review citations against public evidence. Only tests are exported for benchmark scoring. Reference Acceptance Rate during training provides host feedback about the supplied reference; it does not expose the reference contents or the hidden evaluation panel to the test generator. This objective remains distinct from the final Joint Success Function metric.

Independent semantic review. The reviewer receives the following instructions. The final structured response supplies findings, covered requirements, and execution checks; host validation derives acceptance from verified blocking findings.

05 Independent semantic review PROMPT EXCERPTS

```
You review independent regression
tests against the original public
task.  Your workspace contains the
unchanged public baseline plus
generated tests.  You cannot see
generated source repairs or any
hidden oracle.  Inspect the tests
and relevant public interfaces.
Decide whether every assertion
follows from explicit requirements
or clearly established repository
behavior.
```

```
Block only evidenced setup defects,
unsupported assertions or missing
required behavior.  Optional cases
and speculative concerns are
suggestions; host derives acceptance
from blocking findings.  An excerpt
proves its existence, not your
interpretation.
```

```
For a blocking unsupported-assertion
finding, supply falseRejection
describing a concrete allowed
behavior, why it remains allowed
given established compatibility,
whether the assertion is a justified
control, and publicBasis with
taskLines or a repository line
citation.  Omission from the task is
not enough.  If that claim cannot be
established, express the uncertainty
as a suggestion; do not invent a
counterexample or automatically
approve a failing suite.
```

C.3 Recursive Self Improvement

C.3.1 Strategy Schema and Fixed Boundaries

Table 10 documents the defaults and schema bounds of the 11 strategy fields. Experiment configurations can override the defaults. The experimental pair count settings are specified separately in Appendix D.4; the RSI search ranges appear in Appendix C.3.

Controlled optimization space. The fixed $K = 3$ comparison sets both candidate fields to three and excludes them from mutation. The remaining fields allocate the existing inference budget or change reusable guidance; the execution model and its weights, tool implementations, base skill library, execution validator, semantic review protocol, and total task budget stay fixed. Configuration validation imposes additional constraints: differential challenges require at least two rounds, cross execution, semantic review, and corrective feedback. A single proposal changes exactly one declared strategy field. A lesson proposal changes exactly

Table 10: TestHelix strategy fields, defaults, and schema bounds.

Field	Schema default	Schema bounds
`guidance`	Empty string	Reusable prose, at most 2,400 characters
`lessons`	Empty list	At most 20 lessons; at most four per explicit component
`skillRoutes`	Empty list	At most one enabled skill or null per context, seven contexts
`initialCandidates`	2	Integer from 1 to 5, no greater than the maximum
`maxCandidates`	4	Integer from 1 to 5
`maxRounds`	2	Integer from 1 to 3
`agentSteps`	12	Integer from 2 to 20
`repairSteps`	5	Integer from 2 to 10
`contextTokens`	18,000	Integer from 2,000 to 64,000
`toolOutputChars`	10,000	Integer from 512 to 20,000
`reviewMaxOutputTokens`	2,048	Integer from 128 to 16,384

one item while preserving all others. The host validates the schema and rejects embedded task identifiers and code blocks in learned prose; the reflection instructions additionally prohibit copying solutions or hidden patches. The optimizer therefore influences execution through bounded strategy decisions, while model interfaces and validation remain fixed. The effect of coupling between the optimizer and execution model on learned guidance is outside this study's scope.

Lessons and routing. A lesson contains an identifier, one to eight literal retrieval triggers, text of at most 1,000 characters, and optional component and capability tags. Explicit components are planning, preparation, generation, repair, and review. An update renders the text as applicability, action, and observable verification. Retrieval uses the public task, current phase, and host execution checkpoint; it selects at most two applicable lessons within 1,600 characters for an agent turn, or 1,200 for planning. The seven routing contexts are general, algorithm, framework, stateful, performance, CLI, and visual. Routes select from the fixed enabled skill library, and reviewer skills cannot be routed to candidate generation. An empty route list leaves specialist loading to the agent; it does not install a learned routing strategy.

C.3.2 Reusable Guidance and Strategy Reflection

Reusable strategy injection. The candidate receives the following user message only when `guidance` is nonempty. Braces below identify runtime values.

06 Reusable guidance PROMPT EXCERPTS

```
Reusable development-set guidance;
apply only where the original task
supports it:
{guidance}
```

Retrieved lessons use the corresponding phase message and structured lesson text.

07 Retrieved lessons PROMPT EXCERPTS

```
Current {component} methods;
original requirements take priority:
When:  {when}
Action:  {action}
Verify:  {verify}
```

Strategy reflection. The bounded reflection template supplies the strategy adapter with the follow-

ing instructions for revising reusable guidance from execution evidence.

08 Strategy reflection PROMPT EXCERPTS

```
Improve a test-generation policy
from development-set execution
evidence.  Change exactly one policy
component.  For lessons, add, revise
or remove ONE item and preserve
every other item.  Write
applicability, action and an
observable verification condition,
with specific retrieval triggers.
Triggers match literal normalized
phrases in public task instructions,
the current phase/reason, and
workspace evidence:  runnerPresent,
execution command and status, source
repair and task coverage.
```

```
Abstract failure mechanisms across
tasks; do not copy names, paths,
literal expected values, solutions,
task IDs, code, hidden patches, or
instructions to weaken validation.
Preserve the public task as
authoritative.  Do not alter model,
tools, validators, or global budgets.
Explain the observed failure that
motivates the mutation.  Aggregate
training evidence is noisy; it is
not proof of causality.
```

Optimization and evaluation. AutoSaddler uses Claude Opus 4.6 to revise strategies and execute TestHelix during optimization. We freeze the selected strategy's prompts and configured fields, then use this same configuration with GPT 5.6 sol and Kimi K3 for benchmark evaluation without further adaptation.

C.3.3 Objective, Data Roles, and Search Configuration

Objective and isolation. For a task i, let $F_i(\pi) = 1$ when the final test suite submitted by strategy π fails on the original program and passes on its supplied reference repair; otherwise let $F_i(\pi) = 0$. The objective is $\frac{1}{|D|}\sum_{i \in D} F_i(\pi)$, the Reference Acceptance Rate under the evaluation protocol. A missing suite, preparation error, timeout, or unavailable required outcome receives zero in the fixed task denominator. Training does not optimize Joint Success Function or inspect the hidden candidate panel. The reference is accessible only to the host scorer, while the generator receives public task information and bounded execution feedback. The internal three branches produce one final submitted suite: $K = 3$ is not three independent scoring attempts or a pass@3 objective.

Data assignment. The 600 optimization tasks, supplied in the supporting materials, are partitioned into 500 proposal (training) tasks and 100 search development (validation) tasks. All task assignments use seed 42. The proposal tasks provide execution traces for diagnosis and strategy revision. Reference Acceptance Rate on the search development set ranks candidates and can inform subsequent search; it is consequently an optimization signal, not independent validation. Another 100 tasks from the remaining 1,100 in the 1,700 task training pool form a held-out optimization test set for release validation. These three sets and the benchmark test set are disjoint. The assignment changes neither the task pool nor the benchmark denominator.

AutoSaddler adaptation. The AutoSaddler search uses Claude Opus 4.6 for candidate selection, diagnosis and revision, and subsequent reflection through three agent sessions. Opus 4.6 also executes TestHelix on the proposal and development tasks, yielding one strategy search with a fixed execution model. Its evolution DAG records parent relationships, edits, execution outcomes, and reusable lessons. We adapt AutoSaddler (Park et al., 2026) to test generation by invoking the test generation runner in single-reference evaluation mode and mapping each completed task to the binary score above. The adaptation constrains proposals to the strategy schema in Appendix C, checks the changed field, and rejects changes to the evaluator, tools, task inputs, or inference caps.

Optimizer configuration. The corresponding AutoSaddler configuration uses `train_minibatch_size=25`, `train_eval_attempts=1`, `train_epochs=1`, `completed_train_iterations=20`, and `max_candidate_proposals=40`. Both `capability_phase_iterations` and `capability_phase_epochs` are zero; `skip_session0`, `monotonic_full_train_gate`, the adapter's `monotonic_train_gate`, and `use_merge` are false. Selection, mutation, and reflection have separate session timeouts. Optimizer calls, tokens, failures, and elapsed time are separate resource quantities from the generator's inference cap.

Table 11: AutoSaddler settings for the TestHelix search.

Setting	Value and interpretation
Training assignment	500 proposal tasks and 100 search development tasks
Completion schedule	20 completed rounds, 25 distinct proposal tasks per round, one epoch
Evaluation attempts	One final suite per task and strategy; seed 42
Attempt ceiling	40 attempts to complete 20 rounds; retries recorded separately
Optimizer and search execution model	Claude Opus 4.6 for both roles
Session timeouts	Selection 300 s; diagnosis and revision 1,800 s; reflection 600 s
Candidate health check	60 s for strategy parsing, schema checks, and runner loading
Phase and selection	Capability iterations and epochs both zero; steering only; session 0 enabled
Full training gates	Both proposer and engine monotonic training gates disabled
Merge and release	Merge disabled; release validation restricted to one frozen challenger

C.3.4 Initial Strategy, Mutable Fields, and Inference Limits

Initial strategy. The initial strategy eagerly requests three candidates and keeps both candidate count fields fixed at three throughout optimization. It uses two rounds, 12 agent steps, five repair steps, an 18,000 token context limit, a 10,000 character tool output limit, and a 2,048 token review output limit. Cross execution, semantic review, and corrective feedback are enabled, with one differential challenge step and an execution positive fallback. The 12 curated skills remain fixed; guidance, lessons, and skill routes initially are empty. These settings instantiate the initial strategy.

Table 12: Mutable strategy ranges and fixed inference caps.

Field or resource	Values
`maxRounds`	$\{2, 3\}$
`agentSteps`; `repairSteps`	$\{10, 12, 14, 16\}$; $\{3, 5, 7\}$
`contextTokens`	$\{16{,}000, 18{,}000, 24{,}000, 32{,}000\}$
`toolOutputChars`	$\{8{,}000, 10{,}000, 14{,}000\}$
`reviewMaxOutputTokens`	$\{2{,}048, 4{,}096\}$
Textual strategy fields	Guidance, lessons, and skill routes within the existing schema
Task profile caps	48 model calls; 500,000 input and 100,000 output tokens; 900 s, further limited by public task deadlines
Model call concurrency	Two shared concurrent calls
Per call output limits	Generation 8,192 tokens; review initially 2,048 tokens
Execution and timeouts	Two execution repeats; 120 s model timeout; 180 s test timeout
Transport retry allowance	One retry, included in the shared budget and usage ledger

Each proposal changes one of the nine mutable fields; a lesson update changes one lesson while preserving the rest. The six scalar ranges in Table 12 define the scalar search space. Guidance, lessons, and routes obey the schema bounds and evidence restrictions documented in Appendix C. The lower bound of two rounds preserves the differential challenge configuration. Increasing a component limit does not increase the shared task cap: all generation, review, repair, and feedback calls consume the same allowance. Provider reported usage is used when available; otherwise accounting retains conservative reservations. The task's

public runtime limit also applies. For an official agent limit T, the bridge reserves $\min(120, T/4)$ seconds for cleanup and another 20 seconds before passing a runtime cap to the Node process. The profile cannot override that cap: the runner's default $T = 600$ seconds yields a 460 second Node limit despite the 900 second profile setting. The test execution limit is likewise the smaller of 180 seconds and the task's public test timeout. All compared strategies retain the same task deadlines. Execution repeats verify a suite's behavior and do not grant extra independent test generation attempts. The fixed prompts appear in Appendix C.

C.3.5 Screening, Development Tracking, and Independent Release Validation

Each completed round evaluates a parent on 25 previously unused proposal tasks, produces a revision, and evaluates the child afresh on those same tasks. The screen requires a strict increase in successful initial-to-reference transitions and zero regressions among previously successful tasks. Only a child satisfying both conditions receives evaluation on the 100 search development tasks. The initial strategy receives the same development evaluation once. AutoSaddler selects the strategy with the highest Reference Acceptance Rate on the development set under Opus 4.6 and freezes it before release validation. After this search, the RSI configuration in Table 3 uses the same frozen strategy with GPT 5.6 sol and Kimi K3. Neither benchmark model supplies feedback for strategy optimization, and the strategy is not adapted to either model during benchmark evaluation. A development score need not increase for a candidate to be evaluated or retained in the search history.

The release validation protocol compares exactly one frozen challenger with the initial strategy on the separate 100 release validation tasks, using Opus 4.6 to execute both strategies. The promotion rule requires a positive paired difference in Reference Acceptance Rate and a one sided exact McNemar $p \leq 0.05$, with the direction fixed before evaluation. For w challenger only successes and ℓ initial only successes, the test uses the upper binomial tail at w conditional on $w + \ell$ discordant pairs, with probability $1/2$ under the null. With no discordant pairs, there is no evidence for promotion. The protocol reports both discordant counts, both success counts, and 95% and 99% paired bootstrap intervals from 10,000 task resamples using seed 42. Each resample keeps the two outcomes for a task together.

This modest validation set can lack power for small improvements. Failure to pass the gate retains the initial strategy; the validation results must not trigger further adaptive search on those same tasks. The gate is separate from the benchmark scores.

Independent validation results. Table 13 reports the paired comparison on the 100 release validation tasks. Reference Acceptance Rate increases from 48% to 58%, a gain of 10 percentage points. Both strategies succeed on 46 tasks and fail on 40; the remaining 14 tasks comprise 12 new successes and two regressions. The positive gain and one sided exact McNemar $p = 0.006470$ satisfy the release criterion for the frozen strategy.

Table 13: Independent release validation with Opus 4.6. Success counts use 100 paired tasks; confidence intervals describe the Reference Acceptance Rate gain in percentage points (pp).

Initial	After RSI	New successes / regressions	One sided exact McNemar p	95% CI (pp)	99% CI (pp)
48/100	58/100	12 / 2	0.006470	[+3, +17]	[+1, +20]

D Experimental Results and Analysis

The following sections follow the research questions in the main text: baseline performance, patch diversity, test failure mechanisms, and TestHelix component comparisons. Metric definitions appear in Appendix B.6; TestHelix strategies and optimization procedures appear in Appendix C.

D.1 RQ1: Benchmarking Coding Agents

D.1.1 Baseline Configurations

Claude, Kimi, and Qwen models use Claude Code, Kimi Code, and Qwen Code, respectively. GPT 5.6 sol uses Codex, DeepSeek models use DeepSeek Harness, and GLM models use ZCode. Table 2 retains each model family's configuration with the highest Joint Success Function. Table 14 reports the remaining eight configurations, including GPT 5.4 with Terminus-2, sorted by descending Joint Success Function. Earlier versions and model variants support comparisons within families; GPT 5.4 with Terminus-2 provides an additional harness baseline. Baseline comparisons and correlations include all fourteen configurations in Tables 2 and 14.

Native baselines use the latest harness releases available by August 15, 2026, except for ZCode as specified below. They have no experiment-imposed inference budget cap and retain the pub-

lic task execution limits. They do not share the TestHelix inference caps; budget matching applies to the five TestHelix configurations in RQ4. The cutoff releases are Claude Code 2.1.233, Kimi Code CLI 0.36.1, Qwen Code 0.21.12, Codex CLI 0.147.0, DeepSeek Harness 0.1.0-rc.6, and Terminus-2 bundled with Harbor 0.21.0. These are the latest stable releases by the cutoff, except for DeepSeek Harness, whose available release is a release candidate. ZCode uses the August 28, 2026 release. Except for the explicitly labeled `-low` configuration, we use the highest supported reasoning setting: `max` for Claude, Kimi K3, GPT 5.6 sol, DeepSeek, and GLM; `xhigh` for Qwen3.8 Max and GPT 5.4. Kimi K2.7 Code uses mandatory thinking, and Qwen3.7 Plus uses thinking mode without an additional thinking-token cap.

Table 14: **Additional coding agent configurations on TestPrism.**

Model	Reference	Candidates			Coverage		Joint
	RAR	VAR	IRR	H	$\Delta\mathcal{C}$	EC	**JSF**
Kimi K2.7 Code	51.67%	75.33%	55.33%	63.80%	78.33%	49.67%	21.67%
Qwen3.7 Plus	48.67%	81.00%	48.67%	60.80%	72.67%	47.67%	19.67%
Claude Opus 5	55.00%	77.67%	74.33%	75.96%	71.67%	47.33%	19.00%
GPT 5.6 sol -low	35.33%	50.33%	81.67%	62.28%	58.33%	45.67%	14.67%
DeepSeek V4.1 Flash	39.67%	49.67%	78.00%	60.69%	65.33%	44.67%	13.00%
DeepSeek V4 Flash	37.00%	43.00%	83.67%	56.81%	61.33%	45.33%	11.67%
GLM 5.2	39.33%	44.33%	90.33%	59.47%	60.67%	42.67%	10.00%
GPT 5.4	23.67%	39.33%	74.67%	51.52%	56.67%	39.33%	8.33%

D.1.2 Metric Correlations

RQ1 computes Pearson correlations across the aggregate scores in Tables 2 and 14. Each of the fourteen coding agent configurations contributes one paired observation with equal weight. We compare $\Delta\mathcal{C}$ with Entry Coverage and relate each to Reference Acceptance Rate and Joint Success Function. Coverage targets and collection conventions follow Appendix B.6.2. We compute H from Valid Acceptance Rate and Invalid Rejection Rate at the precision shown in the tables; its Pearson correlation with Joint Success Function is $r = 0.83$.

Spearman correlations show the same positive associations: $\rho = 0.88$ between the coverage measures, 0.93 and 0.83 with Reference Acceptance Rate, and 0.90 and 0.96 with Joint Success Function, for $\Delta\mathcal{C}$ and Entry Coverage, respectively. These statistics describe variation across configurations evaluated on the same task set; the unit of analysis is a configuration aggregate.

D.2 RQ2: Patch Diversity and Joint Success

Figure 3 shows Joint Success Function and candidate diversity. The score equally weights the six configurations in Table 2 on the same 50 tasks randomly sampled from the 300 benchmark tasks using seed 42. We use the same candidate generation and verifier validation procedure to expand each task's standard ten implementation panel to 15 implementations: the reference, seven additional valid alternatives, and seven invalid candidates. These additions leave the panels used in the baseline and TestHelix comparisons unchanged. The submitted tests and the 50 task subset remain fixed while candidate sets grow under three addition schedules. The valid and invalid sets in the joint criterion are the selected diagnostic candidate sets, with the same labeling and verdict requirements.

Diversity convention. We use the AST-feature Jaccard distance d_{AST} defined in Appendix B.4. For a diagnostic panel S with at least two candidates, diversity is the arithmetic mean of $d_{\mathrm{AST}}(p, q)$ over its distinct unordered pairs; a singleton is assigned zero. For each task, we add candidates uniformly at random without replacement from the label pool required by the addition schedule, using seed 42. The resulting order defines nested candidate prefixes, each retaining the reference. Figure 3(a) computes diversity on these realized prefixes and averages equally over the 50 tasks. This descriptive mean includes pairs across labels. The group admission criteria apply to standard ten-implementation panels; we do not reapply them to intermediate diagnostic subsets. Panel (c) re-expresses the values in panels (a) and (b), pairing them by addition schedule and patch count k. It is a descriptive visualization parameterized by k, rather than an estimate of the relationship between diversity and joint success at a fixed panel size.

Score calculation. For configuration m and task i, let $c_{im}(p)$ indicate a correctly recorded verdict: acceptance for a valid candidate or rejection for an invalid one. Missing outcomes and recognized errors give $c_{im}(p) = 0$. Beyond the valid reference g_i, let n_i^V and n_i^I count the available valid and invalid candidates for task i. Let s_{im}^V and s_{im}^I count configuration m's correct verdicts in the same candidate pool, and let F_{im} indicate its successful initial to reference transition on task i. Under independent uniform selection without replacement of a additional valid candidates and b invalid candidates, let $\mathcal{P}_{a,b}$ contain these candidates and the reference for each task. Averaging uniformly over the six configurations and 50 tasks, the theoretical

expected joint score is

$$\overline{\text{Joint Success Function}}(a,b) = \frac{1}{6 \cdot 50} \sum_{m=1}^{6} \sum_{i=1}^{50} F_{im} \frac{C(s_{im}^V, a)}{C(n_i^V, a)} \frac{C(s_{im}^I, b)}{C(n_i^I, b)}$$

where $C(n, j)$ counts subsets of size j from n elements, with $C(n, j) = 0$ when $j > n$. The total patch count is $k = 1 + a + b$; the initial program is always checked and is not counted as a patch. The calculation requires $a \leq n_i^V$ and $b \leq n_i^I$ for every task. No task is removed as k increases. Each of the six configurations receives equal weight. For these expanded diagnostic pools, $n_i^V = n_i^I = 7$. This expression averages over all permitted subsets. A uniformly sampled addition order yields a realized sequence of prefixes; measurements on a finite collection of such prefixes can differ from the exact subset expectation. The reported score checkpoints use five valid additions ($k = 6$), five invalid additions ($k = 6$), and nine alternating additions ($k = 10$).

Three addition schedules. Write $k = 1 + j$, where j counts added candidates. The valid addition curve uses $(a, b) = (j, 0)$; the invalid addition curve uses $(a, b) = (0, j)$, both for $0 \leq j \leq 7$. Both retain the valid reference. The alternating curve follows the reference with an invalid candidate, then a valid candidate, continuing to alternate: $(a, b) = (\lfloor j/2 \rfloor, \lceil j/2 \rceil)$ for $0 \leq j \leq 14$. Including the reference, its label sequence is valid, invalid, valid, invalid, and so on. At $k = 15$, the alternating schedule contains eight valid and seven invalid implementations. Within each addition schedule, the curve evaluates Joint Success Function at patch count k under its corresponding schedule, without repeating a candidate.

At $k = 1$, the empty product is one and the metric equals the six configurations' mean Reference Acceptance Rate, denoted $\overline{\text{Reference Acceptance Rate}}$. Each additional required verdict can only preserve or remove success, so expected Joint Success Function is nonincreasing along each schedule. Within the expanded fifteen implementation panels, for $0 \leq a \leq 7$ and $0 \leq b \leq 7$,

$$\begin{aligned} \overline{\text{RAR}} &= \overline{\text{JSF}}(0,0) \\ &\geq \overline{\text{JSF}}(a,b) \\ &\geq \overline{\text{JSF}}(7,7). \end{aligned}$$

Mean Reference Acceptance Rate is therefore the upper bound under the weakest criterion, and the full diagnostic panel gives the lower bound among its subsets. This ordering does not establish a universal bound on test quality or isolate the effect of candidate diversity selection.

Mean Joint Success Function starts at 49.33% and reaches 38.10% after five valid additions, 30.49% after five invalid additions, and 24.33% after nine alternating additions. The complete pool endpoints are 37.33%, 29.33%, and 23.33% for the valid, invalid, and alternating schedules. Complete pools include all admitted candidates; comparisons at different panel sizes impose distinct verdict requirements.

All 50 tasks remain in each configuration's denominator. Missing or erroneous required outcomes receive no success credit, following the main evaluation protocol. Consequently, a decrease can reflect either a candidate disagreement or an additional missing or erroneous required observation.

Marginal acceptance and rejection. Across the fourteen baselines in Tables 2 and 14, Reference Acceptance Rate exceeds Joint Success Function by 15.33 to 36.00 percentage points: 53.07% to 74.58% of reference successes fail the joint criterion. In Table 2, Claude Fable 5.1 leads both Reference Acceptance Rate and Joint Success Function, yet achieves joint success on only 84 of its 179 reference successes. Qwen3.7 Plus has the highest Valid Acceptance Rate at 81.00% but the lowest Invalid Rejection Rate at 48.67%; GLM 5.2 has the highest Invalid Rejection Rate at 90.33% but a Valid Acceptance Rate of 44.33%. Their Joint Success Function values are only 19.67% and 10.00%, respectively. Strong acceptance or rejection alone therefore does not establish reliable discrimination across the panel. These aggregate rates describe different behavioral profiles; they do not identify the overlap of valid and invalid candidate failures within reference successes. RQ1 relates coverage to reference and joint success.

D.3 RQ3: Analysis of Test Generation Failures

D.3.1 Annotation Units

A failed trajectory has task-level joint success indicator $S_i = 0$ under the evaluation protocol. Using seed 42, we sample 50 such trajectories from the configuration with the highest Joint Success Function in each of six model families (Table 2), yielding 300 trajectories. Excluding 14 timeouts leaves 286 for expert annotation and analysis. Each

observation comprises a task, model configuration, submitted test suite, and evaluation attempt. The reported proportions use these 286 trajectories as their denominator: 99 coverage gaps (34.6%), 143 faulty checks (50.0%), and 44 execution logic errors (15.4%).

The examples illustrate the mechanism definitions in Figure 4(b) and the behavior of individual test submissions. Disputed candidate labels do not become evidence of an erroneous generated test solely because the generated verdict disagrees with trusted tests.

D.3.2 Expert Annotation and Agreement

Two experts independently annotate all 286 retained trajectories using the seven error subtypes defined below. They inspect task requirements, submitted test suites, initial and reference executions, candidate outcomes, and supporting patch evidence. Cohen's $\kappa = 0.81$ is computed from these independent subtype labels before adjudication. For each disagreement, a third expert joins both experts in discussion to determine the final label. We aggregate the final labels into the three parent categories reported in Section 4.3 and Figure 4(b).

D.3.3 Subtype Boundaries

Coverage Gaps. Missing Inputs concerns omitted inputs, states, configurations, or combinations within an exercised operation. Untested Requirements concerns entire required behaviors or entry points without acceptance checks.

Faulty Checks. Weak Checks and Wrong Criteria distinguish insufficient checks from incorrect or unsupported behavioral criteria. Overfitting enforces incidental implementation details beyond the task contract.

Execution Logic Errors. Fixture & API Errors concern errors in constructing test fixtures or using APIs within the test process. Command & Parsing Errors concern errors in the test's command execution or output collection and parsing. External installation, network, and container failures remain infrastructure issues. Incorrect expectations about candidate behavior belong to Faulty Checks.

D.3.4 Connection to Candidate Panel Evaluation

A direct single reference blind spot requires the same submitted suite to fail on the initial program, pass on the reference, and accept a candidate whose contract violation is supported by local evidence. Missing Inputs and Weak Checks provide examples of this pattern: the same suite passes the reference while accepting an invalid alternative.

Reference failures caused by execution logic errors or other mechanisms cannot explain a decline conditional on reference success without additional paired evidence. Trusted candidate labels guide witness selection but do not establish correctness for every input. These observations identify mechanisms consistent with the aggregate evaluation gaps; they do not establish causal attribution for aggregate model scores or estimate the prevalence of each mechanism across the benchmark.

D.4 RQ4: Evaluating TestHelix

D.4.1 Experimental Controls

Table 3 includes native agent comparators for each model. The baseline comparison appears in Tables 2 and 14. The following protocol defines five TestHelix configurations, denoted A through E in table order. The first four use the initial test generation strategy; the fifth uses a strategy improved through Recursive Self Improvement (RSI), with three test and repair pairs. Strategies remain frozen during benchmark evaluation. Here K counts internal test and repair pairs, independently of the hidden evaluation panel size.

Single. Configuration A uses one test and repair pair ($K = 1$), with initial state checks, an own repair control, and self correction. Its suite qualifies locally when it reproducibly fails on the initial state and passes on its own repair. Peer Cross Validation is disabled.

Heterogeneous Synthesis with random submission. Configuration B uses Heterogeneous Synthesis to develop three pairs ($K = 3$) with distinct synthesis directions. Each pair is developed independently using local execution checks for self correction. Cross execution, independent semantic review, corrective feedback, and suite merging remain disabled. After local correction, one suite is selected uniformly at random from the locally qualified pairs, using seed 42 fixed before evaluation. This configuration never enters peer cross validation, and selection uses no hidden evaluation outcomes. A task with no qualified suite remains in the evaluation set and receives no success credit; it is not replaced or resampled.

Peer Cross Validation. Configuration C retains three pairs and combines the cross execution, se-

mantic review, and corrective feedback operations in Section 3. Peer repairs qualify as witnesses through local consistency checks, with distinct lineage and source content preventing duplicate support. Independent review interprets the task, original source, tests, and bounded execution symptoms without seeing generated repairs. Reviewed peer counterexamples guide correction. When distinct qualified repairs pass the existing suites and both suites pass review, differential challenges search for missing behavioral distinctions. Revised suites undergo fresh execution and review. Selection uses witness support, and compatible suites with common passing witnesses may be merged. The B to C comparison evaluates this complete feedback process, including its selection and merging rules.

Pair count. Configuration D uses five pairs ($K = 5$) with the same peer cross validation process and initial strategy as C. The C to D comparison varies pair count under the same total inference limit. Five pairs appear only in this comparison, without additional random submission or RSI configurations.

RSI. Configuration E uses peer cross validation with three pairs and one improved, frozen test generation strategy. AutoSaddler uses Opus 4.6 for both strategy revision and task execution during optimization; GPT 5.6 sol and Kimi K3 then share the selected strategy for benchmark evaluation without further adaptation. Both `initialCandidates` and `maxCandidates` are fixed at three throughout strategy search, validation, and benchmark evaluation. The remaining guidance and allocation fields can evolve within the same total budget. The RSI increment compares E with C, which uses the initial strategy at the same pair count. The frozen strategy passes the independent Opus 4.6 release validation in Appendix C.3.5. Benchmark tasks and their hidden panels do not enter strategy updates.

Shared controls and submission. Within each model group, all five configurations use the same base model, public task access, evaluator, tools, base skill library, and total inference limits. Strategy search uses training tasks and a disjoint validation gate; both task sets exclude the final benchmark. Learned allocations remain subject to the same total inference limits as the initial strategy. The budget is shared across pairs rather than multiplied by K; all requested pairs begin in the initial round. The run configuration comprises the planning settings, execution repetitions, random seed, pair count, and strategy. Disabling feedback excludes it from generation and selection, including cached evidence and prompts. Configurations A and B require local consistency for submission. Configurations C, D, and E also require semantic approval; if no common peer witness is available, an approved suite with reproducible initial failure and a passing own repair remains eligible. Internal qualification does not provide success credit in the hidden benchmark evaluation.

Reporting and interpretation. All 300 tasks remain in the Reference Acceptance Rate and Joint Success Function denominators. Valid Acceptance Rate measures acceptance of the four valid alternatives, excluding the reference. This rate and Invalid Rejection Rate use the observed candidate denominator within each task and average the defined task rates with equal weight. Coverage uses observed, nonempty measurement denominators; missing coverage is not zero. Within each model group, all five configurations use the same benchmark task set. These configurations share inference limits. Inference consumption includes every role, retry, and correction call; strategy development and benchmark inference are distinct accounting scopes. Equal limits do not imply equal consumed compute.

The A to B comparison evaluates the complete Heterogeneous Synthesis design, which expands both the number of independently developed test and repair pairs and the range of exploration directions. Guided by the branch prompts, the LLM determines concrete task-specific directions, including complementary behaviors and corner cases. Additional pairs provide space to pursue these directions within the same total inference cap. This comparison therefore measures the overall benefit of heterogeneous expansion relative to Single. The B to C comparison evaluates the complete peer cross validation process, including cross execution, semantic review, and corrective feedback. The C to D comparison measures pair count under a fixed budget, and the C to E comparison measures RSI at fixed $K = 3$. The RSI increment is therefore relative to the three pair peer cross validation configuration, not the adjacent five pair row.